



The Open University

M263 Building Blocks of Software

Unit 10

Logic and language

Study guide

The most important material in the unit is covered in Section 2 (with Subsection 1.1 being a necessary preliminary to that).

Subsection 1.2 contains a small case study, which has some associated computer activities that use the WorkPad. You should not spend more than about 10% of your study time for this unit on this subsection and the associated computing work.

There is a separate computer package that you may find of help in connection with Sections 3 and 4. Use of this package is optional.

You may find some of the material in this unit relatively demanding. If so, do not spend a long time trying to do activities that you find difficult. Aim to read all the material in the unit, but if you find some points hard to grasp, do not spend a long time trying to understand them. In particular, do not spend an excessive amount of time studying Sections 3 and 4.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2302 1

Contents

Introduction	4
1 Predicates	4
1.1 Predicates and variables	4
1.2 A turtle in a box: a small case study	8
2 Quantifiers	14
2.1 The universal quantifier	15
2.2 The existential quantifier	18
2.3 Syntax	20
2.4 Using predicates in specifications	23
3 Boolean algebra	26
3.1 Rules of Boolean algebra	27
3.2 Equivalent expressions with quantifiers	32
4 Rules of deduction	34
4.1 Rules of inference	38
4.2 Contradiction	44
Summary of Unit 10	47
Solutions to the exercises	48
Index	52

Introduction

This unit is about the relationship between natural language and formal logic, a topic that we started to consider in Unit 9. The use of formal logic helps people to develop correct software. If the preconditions and postconditions of methods and functions are written using formal logic, their truth values can be checked by a computer. It may then be possible to prove formally that a method behaves as specified.

In practice, the initial specification of a piece of software is usually written in natural language with all the associated ambiguities and hidden meanings. Thus a key skill is learning to move from natural language specifications to more formal ones, and to clarify any hidden meanings and ambiguities. A main theme of the first part of the unit is to start to develop this skill.

Section 1 reviews the idea of a predicate, which was introduced in Unit 7. It also contains a small case study showing how we can use predicates when writing code. Section 2 introduces an important extension to our formal language. Quantifiers, used in connection with predicates, enable us to express formally a much wider range of propositions, including assertions such as “There are no students in region 07 studying both M150 and M263 in 2004”. We introduce quantifiers with the particular purpose of increasing the range of natural language assertions that can be expressed formally.

Section 3 introduces algebraic methods of analysing Boolean expressions. Section 4 discusses rules of deduction which, together with the algebraic methods, enable us to produce a formal proof that a given logical argument is valid.

Sections 3 and 4 develop techniques that help you to understand formal logical proofs. You will only be expected to construct simple examples of such proofs. These techniques are used when software and hardware developers are wishing to formally prove that methods or software components work as specified.

1 Predicates

In this section and the next we will look at some ways of enriching the formal language of propositions, introduced in Unit 9. The term proposition stands for an “assertion which is true or false”, and propositions are used to describe precisely what we are talking about. English — or any language, for that matter — often uses statements that are either true or false. We start by looking again at the idea of a predicate that was introduced in Unit 7. The second part of this section concerns a small case study that uses predicates in code.

1.1 Predicates and variables

In grammar a **predicate** is the part of the sentence that says something about the subject. For example, consider the sentence

Wirren is a dog.

In this, the subject is “Wirren” and the predicate is “is a dog”. Thus in some sense the predicate is a property of the subject. As we saw in Unit 7, the word predicate has a similar role in logic. It is used to refer to a property of a single item or to a relationship between several items. For example, we might have predicates *DOG*, *FATHER* and *SISTERS* with the following meanings.

1. *DOG*(“Fido”) is the proposition stating
Fido is a dog.
2. *FATHER*(“John”, “Michael”) is the proposition stating
John is the father of Michael.
3. *SISTERS*([“Cordelia”, “Regan”, “Goneril”]) is the proposition stating
Cordelia, Regan and Goneril are female siblings.

Note that for *FATHER* the order in which the inputs “John” and “Michael” are given is important — it affects the meaning of the proposition.

In each of the above cases the predicate is used simply to express a particular proposition. The power of a predicate is that we can put a variety of objects in the round brackets following the predicate identifier. For example, once we have established that *DOG*(*name*) means that

name is a dog

we can construct different propositions such as the following.

- *DOG*(“Rover”) : Rover is a dog.
- *DOG*(“Rover”) $\wedge$ *DOG*(“Wirren”): Rover and Wirren are dogs.
- $\neg$ *DOG*(“Tabby”): Tabby is not a dog.

To avoid confusion with variables, we will give the inputs to predicates as string values, using quotation marks.

We will use the notation $P(x_1, x_2, \dots, x_n)$ to mean that the elements of the ordered tuple $(x_1, x_2, \dots, x_n)$ satisfy the predicate P . As in Unit 7, when we do not have specific values for x_i we will refer to both P and $P(x_1, x_2, \dots, x_n)$ as predicates. So we refer to both *DOG* and *DOG*(*name*) as predicates. Once *name* has been given a specific value, as in *DOG*(“Wirren”), we obtain a proposition (stating that Wirren is a dog).

We will usually drop the quotation marks in giving natural language interpretations, as such confusion is unlikely in that context.

In the examples above, each of the predicates is applied to one or more strings. *DOG* requires a single string input, *FATHER* requires an ordered pair of strings, while *SISTERS* has an input that is a sequence containing two or more strings. However, to be of interest we need to associate these strings with entities in the real world. For the predicate *DOG*, the strings might identify pets at a particular veterinary surgery. For *FATHER* and *SISTERS*, the strings might represent members of a particular family, whose family tree is being investigated.

When using predicates there needs to be a **domain of interest**. This is a predefined set, or collection of predefined sets, to which all the objects under discussion belong. This can be finite, the population of a town, say, or infinite, the integers *Int*, say, or a combination of both. Giving the domain of interest enables us to attach meanings to the predicates. Thus if the domain of interest is the Open University database and if we know that student ID codes are strings with a particular format and that the predicate *M2632004* is the property of studying M263 in 2004, we will be able to interpret *M2632004*(“P1234567”) as the proposition asserting that the student with ID code P1234567 is studying M263 in 2004. In practice, the predefined sets in the domain of interest are not normally all given explicitly. However, any predefined sets that are specific to the domain of interest (rather than data sets of standard library classes) should be given explicitly. In examples, activities and exercises the domain of interest will be specified by giving any relevant predefined sets that are not primitive data sets and by giving meanings to all the predicates.

Some texts use the term **universe of discourse** instead of domain of interest.

Aside

Technically by specifying a domain of interest we are indicating an **interpretation** of the predicates. That is, we are considering the predicates in terms of their semantics (meaning) as well as in terms of their syntax. Although it is possible and sometimes useful to work with predicates purely syntactically, it is beyond the scope of this course to do so.

End of Aside

It is possible to regard predicates as functions with output set *Bool*. The input set of the function depends on the particular predicate together with the domain of interest. For example, suppose that *Dogs* is a subset of *N* where *N* is a given set of names in the domain of interest. The predicate *DOG(name)* is then a function with domain *N*, which returns **true** if *name* is in the set *Dogs* and **false** otherwise.

You will have noticed that the examples of predicates given earlier had varying numbers of arguments. Each predicate has an associated **arity** which is the number of input values to which it applies. Thus *DOG(name)* is a unary predicate, that is a predicate with arity one, since it applies to one argument, and *FATHER(name1, name2)* is a binary predicate, that is a predicate with arity two, since it applies to two arguments.

The word **arguments** is often used for the inputs to a predicate (or those of a function).

SISTERS has arity one, since it has a single argument (a sequence of strings).

Activity 1.1

Suppose that we have a set *Birds* containing the names of a number of species of birds, and predicates associated with this set whose interpretations are given below.

SEABIRD(name): *name* is a seabird.

PREDATOR(name): *name* is a predator.

PREY(name): *name* is a prey species.

BIGGER(name1, name2): *name1* is bigger than *name2*.

- (a) Express each of the following propositions in natural language:
 - (i) *SEABIRD*("puffins") $\wedge$ $\neg$ *SEABIRD*("ostriches");
 - (ii) *SEABIRD*("ostriches") $\vee$ *SEABIRD*("fulmars");
 - (iii) (*PREDATOR*("sparrowhawks") $\wedge$ *PREY*("goldcrests")) $\Rightarrow$ *BIGGER*("sparrowhawks", "goldcrests");
 - (iv) *BIGGER*("ostriches", "goldcrests").
- (b) Express the following natural language propositions formally, using the given predicates.
 - (i) Puffins are not predators.
 - (ii) Gannets and fulmars are seabirds but robins are not.
- (c) What is the arity of *BIGGER*?

As above, we will use string arguments given in quotation marks in formal propositions, but will omit the quotation marks in the natural language interpretations

Discussion

- (a)
 - (i) Puffins are seabirds and ostriches are not seabirds.
 - (ii) Either ostriches or fulmars are seabirds.
 - (iii) If sparrowhawks are predators and goldcrests are prey then sparrowhawks are bigger than goldcrests.
 - (iv) Ostriches are bigger than goldcrests.

Note that we have dropped the quotation marks in the natural language interpretation.

- (b) (i) $\neg \text{PREDATOR}(\text{"puffins"})$.
 (ii) $\text{SEABIRD}(\text{"gannets"}) \wedge \text{SEABIRD}(\text{"fulmars"}) \wedge \neg \text{SEABIRD}(\text{"robins"})$.
 (c) *BIGGER* has arity two, since it has two arguments.

To be able to assign truth values to propositions we need to give more details of the domain of interest.

Alternatively, one can draw on prior knowledge, but we shall not take that approach.

Activity 1.2

Suppose that the set *Birds* is as given below, and that we take “is a seabird” to mean “is in the set *Seabirds* given below” (and similarly for the other predicates in Activity 1.1). Give the truth values of each of the propositions in Activity 1.1(a).

Birds = {“gannets”, “puffins”, “fulmars”, “razorbills”, “ostriches”, “sparrowhawks”, “skuas”, “goldcrests”, “robins”, “blackcaps”}
Seabirds = {“gannets”, “puffins”, “fulmars”, “razorbills”, “skuas”}
Predators = {“sparrowhawks”, “skuas”}
Prey = {“goldcrests”, “robins”, “blackcaps”}
Bigger = {(“gannets”, “puffins”), (“robins”, “goldcrests”), (“sparrowhawks”, “goldcrests”), (“ostriches”, “sparrowhawks”)}

Discussion The proposition in Activity 1.1(a)(i) is **true**. “puffins” is given in the set *Seabirds* and “ostriches” is not.

The proposition in (ii) is **true**. “fulmars” does appear in the set *Seabirds*.

Examining the sets above, we see that the proposition in (iii) has the truth value (**true** $\wedge$ **true**) $\Rightarrow$ **true**, which is **true**.

In terms of the sets given above, the proposition in (iv) is **false**. The pair (“ostriches”, “goldcrests”) does not appear in the set *Bigger*.

For the interpretations of the predicates we have given, someone with fuller knowledge of the context will recognise this proposition as actually being true. (Ostriches are bigger than goldcrests!) This example reminds us that if our formal definitions do not accurately reflect a situation then we may get unexpected results.

You might well observe that, with the set *Bigger* as defined, we appear to know the ostriches are bigger than sparrowhawks, and that sparrowhawks are bigger than goldcrests — and so ought to have noticed that ostriches are bigger than goldcrests! But in such an example you should work with the set *Bigger* as given in the question.

We may believe that we have **complete information** about a domain of interest. We would expect that to be the case with an example like that relating to OU students. We would expect every student studying M263 in 2004 to be recorded in stored data, and so there should be no difference between an interpretation in natural language as “the student with ID P1234567 is studying M263 in 2004”, and the appearance of the identifier P1234567 in a set giving all the students recorded in the OU database as studying M263 in 2004. In such a situation, there is no problem with determining truth values by the presence or absence of the appropriate entry in the stored data.

In the situation in Activity 1.2, we have incomplete information. In such a case, the stored data records what is known to be true so far. In such a case, we cannot afford to interpret “not known to be true” as “known to be false”! This complicates the interpretation of propositions, and the allocation of truth values. For convenience, we shall usually assume that

we have complete information. In practice, one may need to take care to differentiate between the lack of a particular entry in stored data and a firm assertion that the corresponding natural language proposition is known to be false.

We said above that the power of a predicate is that it can be applied to many objects. We illustrated this with the predicate $DOG(name)$, where $name$ is a variable. As another example, suppose we have a predicate $FOURLEGS$ that tells us that the object being described has four legs. Then we can write the assertion

If $name$ is a dog then $name$ has four legs

as $DOG(name) \Rightarrow FOURLEGS(name)$, where $name$ is a variable. By the use of variables, we can increase the power of the language of logic. Variables enable us to be both precise and succinct.

When we give interpretations of predicates, these will only make sense if the variables giving the arguments of the predicates are of an appropriate type. We do not want to write nonsense such as $DOG('a') \Rightarrow FOURLEGS('a')$. The domain of interest may contain several sets, and we will require variables to belong to one of these.

This is similar to the situation when we were describing sets by comprehension in Unit 7.

The truth value of a predicate depends on the values of the variables, as is illustrated in the next activity.

Activity 1.3

Suppose that x is in Int , and we have the following predicates.

$R(x)$: $x < 8$.

$S(x)$: $x > 2$.

$T(x)$: $x < 5$.

Find the truth value of the predicate

$$(S(x) \wedge T(x)) \vee \neg R(x)$$

(a) when $x = 0$,

(b) when $x = 4$.

Discussion (a) $R(0)$ is **true**, $S(0)$ is **false**, $T(0)$ is **true**. So $S(0) \wedge T(0)$ is **false**, and $\neg R(0)$ is **false**. Thus with $x = 0$, the given predicate gives a proposition that is **false**.

(b) $R(4)$ is **true**, $S(4)$ is **true**, $T(4)$ is **true**. So $S(4) \wedge T(4)$ is **true**, while $\neg R(4)$ is **false**. Hence with $x = 4$, the given predicate gives a proposition that is **true**.

1.2 A turtle in a box: a small case study

We noted in Unit 9 that predicates can play an important role when designing code involving conditional statements. We look now at an example where such code is needed. The code involves an object of type **Turtle**. We will implement a function that moves a turtle, in a way that is constrained by a rectangle. The move cannot take the turtle across the boundary of the rectangle. If the turtle is inside the rectangle, then it stays inside; if it is outside the rectangle, then it stays outside. The move will “bounce” the turtle off the boundary of the rectangle, by reversing the direction of the

The class **Turtle** was introduced in Unit 5.

turtle's marker attribute if a move one step in the current marker direction would take the turtle onto the boundary of the rectangle.

To implement this function, we will use a new class. This extends **Turtle** by adding four attributes, giving the boundaries of the rectangle. It also has methods to get and set these attributes, and methods to get the attributes of the turtle's marker. Specifications of the class, and of its new methods, are given below.

Recall that a method that returns the value of an attribute is known as a get method. A method that changes the state of an object by providing values for one or more of the attributes is known as a set method.

TurtleBox is straightforward to implement as an extension of **Turtle**. We omit the details

TurtleBox

Class specification

TurtleBox extends Turtle

States The state of an object of type **TurtleBox** is a turtle with four additional features, giving the boundaries of the box. Each of these features is an attribute of type **Int**. They are identified as: *left*, *right*, *bottom* and *top*.

Default The turtle is in the default state for **Turtle**. The default values of the box attributes are: -5 for *left* and *bottom*, 5 for *right* and *top* (see Figure 1.1).

Methods

getLeft() **return in Int**

getRight() **return in Int**

getBottom() **return in Int**

getTop() **return in Int**

getX() **return in Int**

getY() **return in Int**

getDirection() **return in Int**

setBox(l in Int, r in Int, b in Int, t in Int) **state change**

plus all the methods of **Turtle**

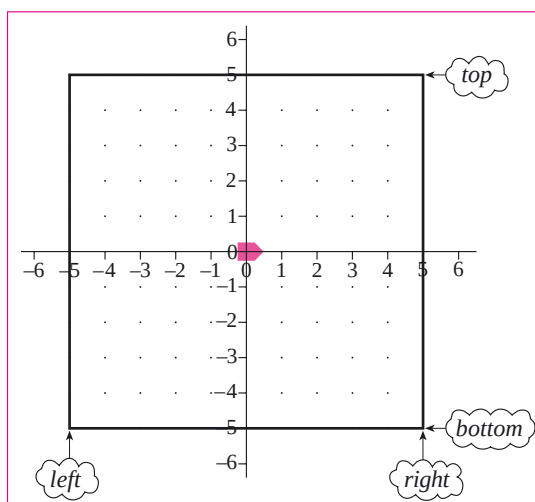


Figure 1.1 The default state of **TurtleBox**.

method *getLeft()* **return in Int**

pre true.

post The returned value is the value of the attribute *left*.

The specifications of *getRight*, *getBottom* and *getTop* are similar to that of *getLeft*. The method *getX* returns the *x*-coordinate of the turtle's position, and *getY* returns its *y*-coordinate.

The method *getDirection* returns an integer representing the turtle's marker direction, as specified below.

method *getDirection()* **return in** **Int**

pre **true**.

post The returned value represents the current direction of the turtle's marker. If this direction is "right(0)" then 0 is returned; if the direction is "up(90)" then 1 is returned; if the direction is "left(180)" then 2 is returned; if the direction is "down(270)" then 3 is returned.

Recall from Unit 5, page 22, that we represent the four possible marker directions by the integers 0, 1, 2 and 3.

method *setBox*(*l in Int, r in Int, b in Int, t in Int*) **state change**

pre $l < -1$ and $1 < r$ and $b < -1$ and $1 < t$.

post The state of the receiver is changed as follows. The attribute *left* is set to *l*. The attribute *right* is set to *r*. The attribute *bottom* is set to *b*. The attribute *top* is set to *t*. The turtle is unchanged.

We now look at how we can implement a function, *FORWARDINBOX*, which moves an object of type **TurtleBox** in a way that bounces off the boundaries of the rectangle. This function is specified below.

function *FORWARDINBOX*(*t in TurtleBox, n in Int*) **return in** **TurtleBox**

pre $n > 0$.

post The turtle that is returned is obtained as follows. A copy of the input turtle is moved *n* times. Before each step, check whether moving one place in the current marker direction would take the turtle onto the boundary of the rectangle. If that is the case, then the marker direction must be changed through 180 degrees before the move is made. Each step moves the turtle one place in the marker direction (adjusted if necessary), and, if the pen is "on", each position visited by the turtle is marked.

If the input turtle is on the boundary of the box, then the first step moves it off the boundary, going in the current marker direction.

Note that *FORWARDINBOX* does not change the state of the input turtle. We can implement this function using the following strategy.

As is the case for all M263 functions.

Repeat the following steps *n* times.

1. If going one step in the current marker direction will hit the edge of the box, then reverse the marker direction.
2. Execute forward(1) (to move the turtle, and add the point visited to the diagram).

Most of this strategy is fairly easy to code. The difficult part is the condition "going one step in the current marker direction will hit the edge of the box". To analyse this formally, it is useful to introduce a related predicate, which will be **true** when this condition is true. We shall call the predicate *moveNotOK*. It requires a single argument, of type **TurtleBox**. The state of such a variable encompasses all the information we need to express the condition: the current marker position; the current marker direction; and the boundaries of the box. The description of *moveNotOK* will depend on whether the current marker position is inside the box. So we will first describe another predicate, *inBox*, which is **true** if the current marker position is inside the box. Again, *inBox* requires a single argument, of type **TurtleBox**.

For a variable *t* of type **TurtleBox**, *inBox*(*t*) will be **true** if four conditions all hold, the first of which is that the current *y*-coordinate of the marker is strictly less than the value of the attribute *top* (which gives the *y*-coordinate of the top edge of the box).

Before we look at formal expression of these predicates, we consider how to implement the function *FORWARDINBOX* so long as the function specified below is available.

```
function MOVENOTOK(t in TurtleBox) return in Bool
pre   true.
post The returned value is moveNotOK(t).
```

This approach of using specified functions that have not yet been implemented is sometimes referred to as “top-down”.

Activity 1.4

Give an implementation of *FORWARDINBOX*. Assume *MOVENOTOK* is available.

Discussion An implementation is given below.

```
function FORWARDINBOX(t,n)
{
  var temp in TurtleBox
  var i in Int
  temp <-- t.clone()
  for (i <-- 1 to n)
  {
    if (MOVENOTOK(temp)) then
    {
      temp.leftTurn()
      temp.leftTurn()
    }
    temp.forward(1)
  }
  return temp
}
```

Activity 1.5

- For a variable *t* of type **TurtleBox**, express formally the condition that the current *y*-coordinate of the marker is strictly less than the value of the attribute *top*.
- Give (in English) the other three conditions that must hold for *t* to lie inside the box (see Figure 1.2). Then express each of these formally, using methods of **TurtleBox**.
- Give an implementation of the function *INBOX* specified below.

```
function INBOX(t in TurtleBox) return in Bool
pre   true.
post The returned value is inBox(t).
```

Discussion

- t*.getY() < *t*.getTop()

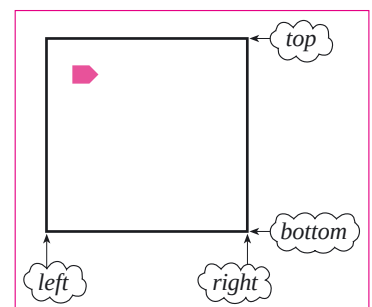


Figure 1.2 A turtle *t* whose position is such that *INBOX*(*t*) is **true**.

(b) We also require the following three conditions to hold. Each is followed by its formal expression.

The attribute *bottom* is strictly less than the *y*-coordinate of the marker.

$t.getBottom() < t.getY()$

The *x*-coordinate of the marker is strictly less than the attribute *right*.

$t.getX() < t.getRight()$

The attribute *left* is strictly less than the *x*-coordinate of the marker.

$t.getLeft() < t.getX()$

(c) An implementation is given below.

```
function INBOX(t)
{
  var b in Bool
  b <-- true
  if ( (t.getLeft() < t.getX())
    /\ (t.getX() < t.getRight())
    /\ (t.getBottom() < t.getY())
    /\ (t.getY() < t.getTop()) ) then
    {b <-- true}
  else
    {b <-- false}
  return b
}
```

For $INBOX(t)$ to be **true**, we need all four of the conditions in (a) and (b) to be **true**, so they are connected by “and”.

Alternatively, one could simply return the expression given in the **if** condition.

This can also be coded using nested **if** statements, but in that case care is needed to ensure that the **else** clause is correctly placed.

Activity 1.6

Let t be of type **TurtleBox**, and suppose that t is inside the box.

(a) A move one place forward would take the turtle onto the edge of the box if the following condition is true. Express this condition formally.

The marker direction is “down(270)” and the *y*-coordinate is 1 more than the attribute *bottom*.

(b) Give three other conditions under which a move one place forward would take the turtle onto the edge of the box. Express each of these formally.

Discussion

(a) Remember that a marker direction of “down(270)” is represented by the attribute *direction* having the value 3. We can express the given condition formally as follows.

$(t.getDirection() == 3) \wedge (t.getY() == t.getBottom() + 1)$

(b) The next move would also take the turtle onto the edge of the box if any of the following is **true**. Each is followed by its formal expression.

The direction is “right(0)” and the *x*-coordinate is 1 less than *right*.

$(t.getDirection() == 0) \wedge (t.getX() == t.getRight() - 1)$

The direction is “up(90)” and the *y*-coordinate is 1 less than *top*.

$(t.getDirection() == 1) \wedge (t.getY() == t.getTop() - 1)$

The direction is “left(180)” and the *x*-coordinate is 1 more than *left*.

$(t.getDirection() == 2) \wedge (t.getX() == t.getLeft() + 1)$

Activity 1.6 considers the value of $moveNotOK(t)$ for the case when the turtle is inside the box. If the turtle is outside the box, then a move one place forward would take it onto the edge of the box under different circumstances, which are given below. (Figure 1.3 shows examples of the marker state of a turtle which is outside the box, and for which $moveNotOK(t)$ is **true**.)

- The direction is “right(0)”, the x -coordinate is 1 less than the attribute *left*, and the y -coordinate lies between *bottom* and *top* (inclusive).
- The direction is “up(90)”, the y -coordinate is 1 less than the attribute *bottom*, and the x -coordinate lies between *left* and *right* (inclusive).
- The direction is “left(180)”, the x -coordinate is 1 more than the attribute *right*, and the y -coordinate lies between *bottom* and *top* (inclusive).
- The direction is “down(270)”, the y -coordinate is 1 more than the attribute *top*, and the x -coordinate lies between *left* and *right* (inclusive).

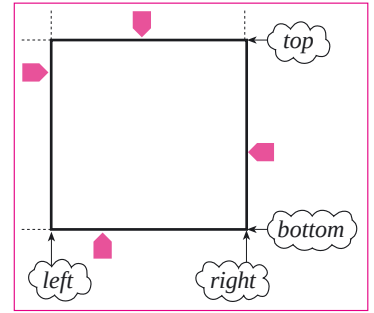


Figure 1.3

Activity 1.7

Let t be of type **TurtleBox**. Express formally that the current state of t satisfies each of the conditions (a)–(d) above.

Discussion Each condition is expressed formally below.

- $(t.getDirection() == 0) \wedge (t.getX() == t.getLeft() - 1) \wedge (t.getBottom() \leq t.getY()) \wedge (t.getY() \leq t.getTop())$
- $(t.getDirection() == 1) \wedge (t.getY() == t.getBottom() - 1) \wedge (t.getLeft() \leq t.getX()) \wedge (t.getX() \leq t.getRight())$
- $(t.getDirection() == 2) \wedge (t.getX() == t.getRight() + 1) \wedge (t.getBottom() \leq t.getY()) \wedge (t.getY() \leq t.getTop())$
- $(t.getDirection() == 3) \wedge (t.getY() == t.getTop() + 1) \wedge (t.getLeft() \leq t.getX()) \wedge (t.getX() \leq t.getRight())$

We can use these ideas to implement the function *MOVENOTOK* specified earlier. (See Exercise 1.2.)

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Use variables in predicates.
- ☐ For a specified domain of interest, find the truth value of a proposition involving predicates where all the variables have been replaced by specific values.
- ☐ Trace code that uses predicates in conditions.

Exercises on Section 1

Exercise 1.1

For the predicates and sets described in Activities 1.1 and 1.2, find the truth value of the following proposition:

$$(PREY(\text{"robins"}) \wedge PREDATOR(\text{"sparrowhawks"})) \Rightarrow BIGGER(\text{"sparrowhawks"}, \text{"robins"}).$$

Exercise 1.2

Give an implementation of the function *MOVENOTOK*. Assume *INBOX* is available.

2 Quantifiers

For the domain of interest described in Activity 1.1, suppose that we want to express propositions such as the following.

1. Ostriches are bigger than all the seabirds.
2. There is a seabird that is a predator.

To express such propositions formally, we need to enhance our formal language. We do so by introducing some conditions that may be attached to variables being used. There are two restrictions that we can make, both called **quantifiers**.

Consider the expression $SEABIRD(x) \Rightarrow BIGGER(\text{“ostriches”}, x)$, where x is an arbitrary bird. This means “if x is a seabird, then ostriches are bigger than x ”. This is clearly related to the first example above. As it stands, this expression is a predicate, whose truth value depends on the value of the variable x . Now the assertion in 1 above is a specific proposition, containing nothing corresponding directly to the variable x . This specific proposition is related to the predicate $SEABIRD(x) \Rightarrow BIGGER(\text{“ostriches”}, x)$ — it asserts that for every particular value of x , the predicate gives a proposition that is **true**. We need to be a little more specific than this though. We need to say what values of x are included in “every particular value of x ” here. In our formalism, we do this by stating what set the variable x must come from. A formal expression of the first proposition above is given below.

$$\forall x \text{ in } Birds[SEABIRD(x) \Rightarrow BIGGER(\text{“ostriches”}, x)]$$

This introduces a new symbol, $\forall$, which may be read as “for all”. A literal interpretation of this formal expression would be: “For all birds, x , if x is a seabird, then ostriches are bigger than x ”. But an expression of this in everyday English will not refer to the variable, x .

The second assertion above is again a specific proposition. This is related to the predicate $SEABIRD(x) \wedge PREDATOR(x)$. This predicate again concerns an arbitrary bird represented by x , and states that x is both a seabird and a predator. The second proposition above asserts that there is at least one bird for which the predicate is true. To express this proposition formally, we need to introduce another new symbol, $\exists$, which can be read as “there exists”. So the formal proposition below asserts that there exists a bird in the set *Birds* that is both a seabird and a predator.

$$\exists x \text{ in } Birds[SEABIRD(x) \wedge PREDATOR(x)]$$

The literal interpretation of each of the quantifiers corresponds to the symbolism in a way that helps with remembering their meanings. If the first letter of “Exists” is inverted, then it looks like the symbol $\exists$. Similarly the ‘A’ in “All” can be inverted, and then looks like $\forall$.

We discuss the interpretation and use of these two new symbols more fully in this section. With the specific sets defined in Activity 1.2, the first of the propositions given above is **false**, while the second is **true**.

See Exercise 2.1.

2.1 The universal quantifier

The **universal quantifier**, denoted by $\forall$, can be read as “for all”. The variable following the quantifier must come from a set within the domain of interest. The notation we use shows which set in the domain of interest the variable must come from. For example, if the domain of interest includes both numbers and strings, we show that x is an integer by writing $\forall x \text{ in } Int$.

Suppose that the domain of interest contains the sets *Char* and *Int*, and a dictionary *Dict* containing a specified set of strings. Suppose also that we have the following predicates, where *word* comes from the dictionary *Dict*.

FIRST(*word*, *c*): *word* begins with the character *c*.

SECOND(*word*, *c*): *word* has the character *c* in the second position.

LAST(*word*, *c*): *word* ends with the character *c*.

HAS(*word*, *number*): *word* has at least *number* characters.

LETTER(*c*): the character *c* is an upper- or lower-case letter.

Note that in these definitions, *c* is a variable, *not* the specific character ‘c’.

We shall use these predicates regularly throughout this section. So far as the interpretations of the predicates are concerned, we do not need to know exactly what words are contained in *Dict*. To give the truth values of propositions concerning *Dict*, though, we do need to say exactly what is in the dictionary. We shall use the particular small dictionary given below.

This example was used in Unit 9.

Dict = {“aardvark”, “aardvogel”, “qanat”, “qintar”, “quadrans”, “syzygy”, “zymurgy”}

If an expression contains a variable and that variable is not restricted by a quantifier, we call the variable a **free variable**. A variable that is restricted by a quantifier is known as a **bound variable**. In giving formal expressions corresponding to statements in English, one would usually expect all variables to be bound. On the whole, natural language does not use free variables.

Even a free variable should belong to a specified set in the domain of interest.

Using the predicates above, we can express formally the proposition that “All words in the dictionary *Dict* have at least one character.” as follows.

$$\underbrace{\forall \text{word in } Dict}_{\text{binding term}} \underbrace{[HAS(\text{word}, 1)]}_{\text{scope}}$$

In this expression, the **binding term** consists of the quantifier, variable and set to which the variable belongs, while the following part of the expression, in square brackets, is known as the **scope** of the quantifier. The square brackets show the extent of the relationship between the quantifier and the variable that it binds. The variable is bound by the quantifier within the scope of the quantifier.

The truth value of the proposition $\forall x \text{ in } U[p(x)]$ is determined as follows.

If $p(x)$ is **true** for every value of x in U , then the quantified proposition is **true**.

If you can find a single value of x in U for which $p(x)$ is **false**, then the quantified proposition is **false**.

So long as U is a small finite set, we can determine the truth value of a quantified proposition by checking the truth value of its scope for each member of the set in turn.

Activity 2.1

For each of the following propositions, give the identifier, binding term, and scope of any variables:

- (a) $\forall \text{word in Dict}[LAST(\text{word}, 'y')]$;
- (b) $\forall \text{word in Dict}[\forall c \text{ in Char}[FIRST(\text{word}, c) \Rightarrow LETTER(c)]]$;
- (c) $LETTER(c) \Rightarrow (c \in Char)$.

Discussion

- (a) There is one variable with identifier *word*. The binding term is $\forall \text{word in Dict}$ and the scope is $[LAST(\text{word}, 'y')]$.
- (b) There are two variables with identifiers *word* and *c*. The binding term for *word* is $\forall \text{word in Dict}$ and it has scope $[\forall c \text{ in Char}[FIRST(\text{word}, c) \Rightarrow LETTER(c)]]$.

The binding term for *c* is $\forall c \text{ in Char}$ and it has scope $[FIRST(\text{word}, c) \Rightarrow LETTER(c)]$.

Notice that the variables have different scopes.

- (c) There is one variable with identifier *c*. It has no binding term and no scope — *c* is a free variable here. Notice that $c \in Char$ is not a binding term. It is a proposition about *c*. (To be precise, the given expression is a predicate, not a proposition.)

Note that 'y' is a specific character — not a variable ...
... while *c* is a variable — not a specific character!

When providing an interpretation in English of a formal proposition, one may need to go through two stages. First, one can translate the formal proposition rather literally. Then one can see how to express the idea in more natural English. It is useful to do this, if only to help when one comes to the task of moving the other way, from English to formalism. In that case, one is likely to start with everyday English.

For example, consider the following proposition.

$$\forall \text{word in Dict}[\neg FIRST(\text{word}, 'b')]$$

This can be read literally like this.

For all words in the dictionary, it is not the case that the word begins with the character 'b'.

A more succinct expression of this is:

For any word in the dictionary, the word does not begin with 'b'.

But everyday English would express this proposition in a form such as:

No word in the dictionary starts with 'b'.

As another example, consider the proposition (2.1) below.

$$\forall \text{word in Dict}[FIRST(\text{word}, 'a') \Rightarrow \neg LAST(\text{word}, 'z')] \quad (2.1)$$

A literal interpretation of this might be:

For all words in the dictionary, if the word begins with 'a', then it does not end with 'z'.

Now consider the proposition (2.2) below.

$$\forall \text{word in Dict}[\neg (FIRST(\text{word}, 'a') \wedge LAST(\text{word}, 'z'))] \quad (2.2)$$

A literal translation of this might be

For all words in the dictionary, it is not the case that the word begins with ‘a’ and ends with ‘z’.

Think for a moment about how each of these propositions might be expressed in more natural English.

The proposition (2.1) might be expressed as follows.

No word in the dictionary begins with ‘a’ and ends with ‘z’.

For the proposition (2.2), exactly the same interpretation can be given. The fact that these two propositions have a common interpretation results from the structure of the formal propositions, and is not dependent on the interpretation of the particular predicates involved. We say that propositions (2.1) and (2.2) are equivalent, and will examine the idea of equivalence more formally in the next section.

In Unit 9, we gave precedence to $\neg$ when parsing propositions. We will do the same for expressions involving predicates and quantifiers. For example,

$\forall \text{word in Dict}[\neg \text{LAST}(\text{word}, 'a')]$ means the same as

$\forall \text{word in Dict}[\neg(\text{LAST}(\text{word}, 'a'))];$

and

$\neg \forall \text{word in Dict}[\text{FIRST}(\text{word}, 'q')]$ means the same as

$\neg(\forall \text{word in Dict}[\text{FIRST}(\text{word}, 'q')]).$

To avoid confusion, predicate identifiers should not include any of the special logical symbols, such as $\neg$ or $\wedge$.

Activity 2.2

For each proposition below, express it in natural language, and give its truth value (where *Dict* is the particular small dictionary given on page 15):

(a) $\forall \text{word in Dict}[\neg \text{LAST}(\text{word}, 'a')];$

(b) $\neg \forall \text{word in Dict}[\text{FIRST}(\text{word}, 'q')].$

When interpreting these propositions, remember that we give precedence to $\neg$.

Discussion

(a) A direct translation might be:

For all words in the dictionary it is not the case that the word has ‘a’ as its last letter.

This can be expressed as:

No word in the dictionary has ‘a’ as its last letter.

To determine the truth value of this, we need to look at the dictionary *Dict*. Examining each word in *Dict* in turn, we see that none of them has ‘a’ as its last letter. So this proposition is **true**.

(b) A literal translation might be:

It is not the case that for all the words in the dictionary the word starts with ‘q’.

This might be expressed as follows.

Some words in the dictionary do not start with ‘q’.

Dict does contain some words that do not start with ‘q’; “aardvark”, for example. So this proposition is **true**.

2.2 The existential quantifier

The **existential quantifier**, which is denoted by $\exists$, is literally read as “there exists”. Alternative ways of expressing the same idea include “there is”, “for some” or “there is some”. As with $\forall$, the existential quantifier precedes a variable that must come from a specified set in the domain of interest.

We use the terminology “binding term” and “scope” for $\exists$ just as with $\forall$. As before, the variable will belong to a given set, shown by **in** followed by the set name, and the scope of the quantifier is indicated by square brackets.

Using predicates defined earlier, we can construct the following proposition.

$$\exists \text{word in Dict}[FIRST(\text{word}, 'q')]$$

This may be read in natural language as

There is a word in the dictionary beginning with ‘q’.

Using the two quantifiers, we can express complex ideas. This can lead to quite complicated formal expressions.

Activity 2.3

In the proposition below, identify the variables, and the binding and scope of each variable.

$$\forall \text{word in Dict}[HAS(\text{word}, 2)] \wedge \forall c \text{ in Char}[\exists \text{word in Dict}[FIRST(\text{word}, c)]]$$

Discussion There are three variables.

word has binding $\forall \text{word in Dict}$ and scope $[HAS(\text{word}, 2)]$.

c has binding term $\forall c \text{ in Char}$ and scope $[\exists \text{word in Dict}[FIRST(\text{word}, c)]]$.

Then there is a second variable with identifier *word*. This has binding $\exists \text{word in Dict}$ and scope $[FIRST(\text{word}, c)]$.

You will have noticed that two of the variables have the same identifier *word*. We can consider them as different variables, however, because their scopes are different. Although this proposition is technically correct, it would be better to give these two variables different identifiers.

The truth value of the proposition $\exists x \text{ in } U[p(x)]$ is determined as follows.

If you can find any value of *x* in *U* for which *p(x)* is **true**, then the quantified proposition is **true**.

If *p(x)* is **false** for every value of *x* in *U*, then the quantified proposition is **false**.

The next few activities illustrate moving from formal propositions using quantifiers to their natural language interpretation.

Activity 2.4

Express each of the propositions below in natural language. Give its truth value (where *Dict* is the particular dictionary given on page 15).

(a) $\exists \text{word in Dict}[SECOND(\text{word}, 'u')]$.

(b) $\exists \text{word in Dict}[\neg SECOND(\text{word}, 'u')]$.

(c) $\neg \exists \text{word in Dict}[SECOND(\text{word}, 'u')]$.

Discussion Each time we will give a literal interpretation first, then express it a more normal way, then give its truth value.

- (a) There exists a word in the dictionary that has ‘u’ as its second letter.

There is a word in the dictionary with ‘u’ as its second letter.

This is **true**: “quadrans” is in *Dict* and has ‘u’ as its second letter.

- (b) There exists a word in the dictionary that does not have ‘u’ as its second letter.

There is a word in the dictionary without ‘u’ as its second letter.

This is **true**; for example, “aardvark” does not have ‘u’ as its second letter.

- (c) It is not the case that there exists a word in the dictionary whose second letter is ‘u’.

No word in the dictionary has ‘u’ as its second letter.

This is **false**: it is the negation of the proposition in part (a).

Activity 2.5

Express each of the propositions below in natural language. Give the truth value when *Dict* is the particular dictionary given on page 15.

- (a) $\exists \text{word in } Dict[FIRST(\text{word}, 'a')] \wedge \exists \text{word in } Dict[FIRST(\text{word}, 'q')]$.
 (b) $\exists \text{word in } Dict[FIRST(\text{word}, 'a') \wedge FIRST(\text{word}, 'q')]$.

Discussion The proposition in (a) is potentially confusing, since it uses the same identifier for the variable in both parts of the expression, and these variables can identify different words, because the two quantifiers here have different scopes. In (b), the variable appears twice within the scope of the same quantifier, and so must relate to the same word in the two cases.

- (a) There exists a word in the dictionary whose first letter is ‘a’, and there exists a word in the dictionary whose first letter is ‘q’.

The dictionary contains a word beginning with ‘a’ and one beginning with ‘q’.

This is **true**: *Dict* contains, for example, “aardvark” and “qanat”.

Here, the words may be different.

- (b) There exists a word in the dictionary that both starts with the letter ‘a’ and starts with the letter ‘q’.

This is **false**: no word can start with two different letters!

Here, the words must be the same.

When expressing propositions formally, there are ways in which one can make them easier to interpret. For example, the proposition (2.3) below is syntactically correct.

$$\exists w \text{ in } Dict[FIRST(w, 'a') \wedge \exists v \text{ in } Dict[FIRST(v, 'q')] \wedge LAST(w, 's')] \quad (2.3)$$

It can be interpreted as follows.

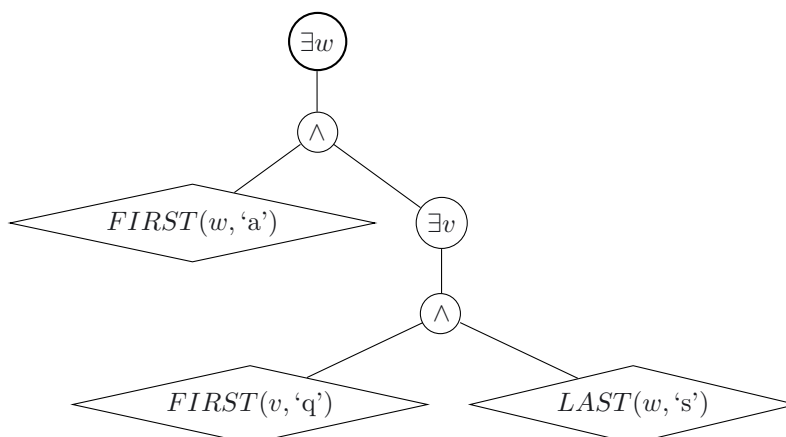
There is a word in the dictionary beginning with ‘a’ and ending with ‘s’ and there is a word in the dictionary beginning with ‘q’.

However, it would be much clearer to keep the scopes separate, as below.

$$\exists w \text{ in } Dict[FIRST(w, 'a') \wedge LAST(w, 's')] \wedge \exists v \text{ in } Dict[FIRST(v, 'q')]$$

2.3 Syntax

Expressions in predicate calculus have a syntax, just as do Boolean expressions. Essentially, we need the expression inside the square brackets following a quantifier to be syntactically correct as a Boolean expression. We can draw parse trees for quantified expressions. Each quantifier is placed in a node with the variable it quantifies. The scope of the quantifier is then the subtree below that node. For example (given that the variables v and w come from the set $Dict$), the parse tree on the right represents the following expression.



$\exists w \text{ in } Dict[FIRST(w, 'a') \wedge \exists v \text{ in } Dict[FIRST(v, 'q') \wedge LAST(w, 's')]]$

Using both quantifiers, and the Boolean connectives, we can form complicated propositions. When doing so, one needs to take care about the syntax. Firstly, one needs to ensure that the expression has valid syntax. Also, it is important to get the brackets correctly placed in the expression. Quite small rearrangements of bracketing can make significant changes to the meaning of a proposition, as illustrated in Activity 2.5, for example.

Activity 2.6

Express each proposition in natural language twice. The first version should be a more or less direct translation. The second should express the meaning in a more normal manner.

- (a) $\forall \text{word in } Dict[\exists c \text{ in } Char[LETTER(c) \wedge FIRST(\text{word}, c)]]$.
- (b) $\exists c \text{ in } Char[\forall \text{word in } Dict[LETTER(c) \wedge FIRST(\text{word}, c)]]$.

Discussion

- (a) For all the words in the dictionary, there exists a character that is a letter and for which the word starts with this letter.

Every word in the dictionary starts with some letter.

- (b) There exists a character such that for all the words in the dictionary, the character is a letter and the word has that letter as its first character.

Every word in the dictionary starts with the same letter.

If we compare the interpretation of the two propositions in Activity 2.6, we see that the second is a much stronger assertion than the first. Where the two quantifiers are mixed, the order in which they appear is crucial to the meaning of the proposition. Again, when negation is used, particular care is needed to introduce it in the appropriate place.

Activity 2.7

For each of (a)–(d), express the proposition in natural language twice. The first version should be more or less a direct translation. The second should express the meaning in a more normal manner. Give the truth value of each proposition, where *Dict* is as given on page 15.

- (a) $\neg \exists \text{word in } Dict[FIRST(\text{word}, 'a')]$.
- (b) $\forall \text{word in } Dict[\neg FIRST(\text{word}, 'a')]$.
- (c) $\neg \forall \text{word in } Dict[FIRST(\text{word}, 'a')]$.
- (d) $\exists \text{word in } Dict[\neg FIRST(\text{word}, 'a')]$.

Discussion We first give the natural language interpretations.

- (a) It is not the case that there exists a word in the dictionary that has ‘a’ as its first character.

No word in the dictionary starts with ‘a’.

- (b) For every word in the dictionary it is not the case that the word has ‘a’ as its first character.

No word in the dictionary starts with ‘a’.

- (c) It is not the case that all words in the dictionary have ‘a’ as their first character.

Some words in the dictionary do not start with ‘a’.

- (d) There exists a word in the dictionary that does not have ‘a’ as its first character.

Some words in the dictionary do not start with ‘a’.

The propositions in (a) and (b) are **false**: for example, *Dict* contains the word “aardvark”, which does start with ‘a’. The propositions in (c) and (d) are **true**: for example, “qanat” is in *Dict*, and does not start with ‘a’.

In Activity 2.7, we can see that the propositions in (a) and (b) have the same meaning. So do the propositions in (c) and (d). This equivalence of meaning is not accidental, and does not depend on the particular context being discussed. In both cases, there is an underlying equivalence of propositions. We shall discuss these in Section 3.

So far we have been rewriting symbolic expressions in natural language. We also need to work the other way round, that is, rewrite natural language statements symbolically using predicates and quantifiers.

Activity 2.8

Write each of the following expressions formally using quantifiers and the predicates on page 15.

- (a) Every word in the dictionary has at least two characters in it.
- (b) Every word in the dictionary ends with a letter.
- (c) None of the words in the dictionary starts with ‘b’ and ends with ‘y’.

Discussion The first step in expressing these propositions formally is to recognise which quantifiers will be needed. “Every” suggests that “for all” will be required. “No” suggests that “it is not the case that there exists ...” (or its equivalent: “for all... it is not the case that ...”).

(a) $\forall \text{word in Dict}[HAS(\text{word}, 2)]$

(b) Where a formal version is not readily seen, you may find it helpful to expand the natural language into a version of the proposition that corresponds more closely to the formal language. Here, one might rewrite the given proposition like this.

For all words in the dictionary, there exists a character that is a letter, and is the last character of the word.

This suggests the following formal version.

$$\forall \text{word in Dict}[\exists c \text{ in Char}[LETTER(c) \wedge LAST(\text{word}, c)]]$$

(Note that if you have the quantifiers in the reverse order, you have the wrong answer! The proposition

$$\exists c \text{ in Char}[\forall \text{word in Dict}[LETTER(c) \wedge LAST(\text{word}, c)]]$$

says that there exists a letter such that all words in the dictionary have that letter as their last character. That is,

Every word in the dictionary ends with the same letter.)

(c) Perhaps the most likely way of reading this more formally is the following. It is not the case that there exists a word in the dictionary that both starts with ‘b’ and ends with ‘y’. This leads to the formal expression below.

$$\neg \exists \text{word in Dict}[FIRST(\text{word}, 'b') \wedge LAST(\text{word}, 'y')]$$

However, this can equally well be expressed formally as below.

$$\forall \text{word in Dict}[\neg (FIRST(\text{word}, 'b') \wedge LAST(\text{word}, 'y'))]$$

As observed immediately after Activity 2.6, changing the order of $\forall$ and $\exists$ changes the meaning of the proposition.

2.4 Using predicates in specifications

Software is verifiable if it can be shown that it meets the specifications. This requires that specifications are unambiguous and clearly stated. For this reason it is useful if specifications can be written down formally. We will look at some examples to see what this entails. You will not normally be expected to specify functions or methods with this level of formality. These examples and activities are intended to give a flavour of such formality.

EXAMPLE 2.1

We will start by looking at a simple function on integers.

```
function ADDONE(x in Int) return in Int
pre   true.
post  The returned value is  $x + 1$ .
```

Here the precondition cannot be made more precise or formal but the postcondition, although precise, is not expressed in a way suitable for machine checking. That would require a Boolean expression, and for this we need a way of referring to the returned value. We can do this using the function name. Thus to write the specification down formally we express the postcondition as below.

```
function ADDONE(x in Int) return in Int
pre   true.
post  ADDONE(x) ==  $x + 1$ .
```

Formal specification languages, such as Z, use their own notation in such a context.

To determine what is available to express the formal specification of a function, look at the signature. We shall express the precondition and postcondition using the type functions or methods of the types that appear in the signature. Thus, in the above example the specification makes use of the infix operation $+$ of the data type **Int**.

In doing this, we assume that these have been correctly implemented.

Activity 2.9

Write the following specification formally.

```
function FIRST(s in mString) return in Char
pre   s has at least one character.
post  The returned value is the first character of the string.
```

Discussion In this case we need to write both the precondition and postcondition formally.

The precondition simply needs to check that the length of the input string *s* is non-zero. It can be expressed as *s.length()* > 0.

The postcondition needs to check that the correct character has been returned. The first character of *s* is given by *s.charAt*(1).

So we can express this function specification formally as follows.

```
function FIRST(s in mString) return in Char
pre   s.length() > 0.
post  FIRST(s) == s.charAt(1).
```

The precondition of *FIRST* ensures that in the expression *s.charAt*(1) the precondition of *charAt* is respected.

It can be quite difficult to express the precondition and postcondition formally. We may need to use quantifiers. The next activity concerns the function *MAX*, specified informally below.

function *MAX*(*v* in **Vector of Int**) **return** in **Int**

pre *v* contains at least one element.

post The returned value is the largest number in *v*.

Activity 2.10

- Suppose that *v* is a non-empty vector. Let *A*(*v*) be the set of integers that give the indices of positions in the vector *v*. Using comprehension, express the set *A*(*v*) formally, using methods of the class **Vector**.
- Write down a formal proposition stating that *MAX*(*v*) appears as an element of the vector *v*.
- Write down a formal proposition stating that *MAX*(*v*) is at least as large as any element in the vector *v*.
- Express the specification of the function *MAX* formally.

In answering parts (b), (c) and (d), you can use quantification over the set *A*(*v*).

Discussion Recall that *v.at*(*i*) returns the *i*th element of the vector *v*.

- The set is: $A(v) = \{i \text{ in } \mathbf{Int} : (1 \leq i) \wedge (i \leq v.size())\}$
- We need to assert that there exists an integer (say *i*), for which *MAX*(*v*) is the element of the vector *v* at position *i*. Hence the proposition is: $\exists i \text{ in } A(v)[v.at(i) == MAX(v)]$.
- We need to assert that, for every integer giving a position in the vector *v*, the element of *v* at that position is less than or equal to *MAX*(*v*): $\forall i \text{ in } A(v)[v.at(i) \leq MAX(v)]$.
- function** *MAX*(*v* in **Vector of Int**) **return** in **Int**
pre *v.size*() ≥ 1.
post $\exists i \text{ in } A(v)[v.at(i) == MAX(v)] \wedge \forall j \text{ in } A(v)[v.at(j) \leq MAX(v)]$

An integer *i* is the index of an item in *v* so long as *i* lies between 1 and the size of *v*, inclusive.

Note that if an integer *i* is in *A*(*v*) then a method call *v.at*(*i*) will respect the precondition of *at*.

It is not essential to use different variables *i* and *j* as the scopes are separate, but doing so helps clarity.

As is illustrated by this last activity, drawing up formal specifications requires considerable skill and attention to detail. Much of this comes with practice and experience.

Objectives for Section 2

After studying this section you should be able to do the following.

- Given a domain of interest, predicates, and specific sets if necessary:
 - identify the binding term and scope of each variable in a formal proposition involving quantifiers;
 - give an interpretation in natural language of a formal proposition involving quantifiers;
 - for a proposition expressed in natural language, give a formal proposition (possibly involving quantifiers) with the same meaning;
 - determine the truth value of a proposition involving quantifiers.
- Using the type functions and methods of the types in the signature, give a formal specification of a function.

Exercises on Section 2

Exercises 2.1 and 2.2 both refer to the following domain of interest (introduced in Section 1).

The set *Birds* contains the names of a number of species of birds, and certain predicates associated with this set have the interpretations given below.

SEABIRD(*name*): *name* is a seabird.

PREDATOR(*name*): *name* is a predator.

PREY(*name*): *name* is a prey species.

BIGGER(*name1*, *name2*): *name1* is bigger than *name2*.

When assigning truth values, assume that the set *Birds* is as given below, and that we take “is a seabird” to mean “is in the set *Seabirds* given below” (and similarly for the other predicates).

Birds = {“gannets”, “puffins”, “fulmars”, “razorbills”, “ostriches”, “sparrowhawks”, “skuas”, “goldcrests”, “robins”, “blackcaps”}

Seabirds = {“gannets”, “puffins”, “fulmars”, “razorbills”, “skuas”}

Predators = {“sparrowhawks”, “skuas”}

Prey = {“goldcrests”, “robins”, “blackcaps”}

Bigger = {(“gannets”, “puffins”), (“robins”, “goldcrests”), (“sparrowhawks”, “goldcrests”), (“ostriches”, “sparrowhawks”)}

Exercise 2.1

Write each of the following propositions in natural language, and give their truth values.

- (a) $\forall \textit{name} \text{ in } \textit{Birds} [\textit{SEABIRD}(\textit{name}) \vee \textit{PREY}(\textit{name})]$.
- (b) $\forall \textit{name} \text{ in } \textit{Birds} [\textit{SEABIRD}(\textit{name}) \Rightarrow \textit{BIGGER}(\text{“ostriches”, } \textit{name})]$.
- (c) $\exists \textit{name} \text{ in } \textit{Birds} [\textit{SEABIRD}(\textit{name}) \wedge \textit{PREDATOR}(\textit{name})]$.
- (d) $\exists \textit{name} \text{ in } \textit{Birds} [\forall \textit{sea} \text{ in } \textit{Birds} [\textit{SEABIRD}(\textit{sea}) \Rightarrow \textit{BIGGER}(\textit{name}, \textit{sea})]]$.

Exercise 2.2

Express each of the following propositions formally, using the set *Birds* and predicates in Exercise 2.1.

- (a) All seabirds are bigger than goldcrests.
- (b) There is no bird that is both a predator and a prey species.
- (c) Some birds are seabirds and some birds are predators.

Exercise 2.3

- (a) Suppose that *s* is a non-empty string. Let *A*(*s*) be the set of integers that give the indices of positions in the string *s*. Using comprehension, express the set *A*(*s*) formally, using methods of the class **mString**.
- (b) Express the following function specification formally. In doing this, you can use the set *A*(*s*).

function *FIRSTOCCUR*(*s* in **mString**, *c* in **Char**) **return** in **Int**
pre *s* is not empty and the character *c* occurs in the string *s*.
post The returned value is the first position in the string *s* where the character *c* occurs.

3 Boolean algebra

In Unit 9 we introduced Boolean expressions and used truth tables to show the truth value of an expression for all possible combinations of Boolean values of the atoms it contains. A problem with that approach is that truth tables get very large if there are more than 3 or 4 atoms. (Truth tables for 5 atoms will have 32 rows). Hence it is useful to have techniques at our disposal that enable us to manipulate such expressions algebraically. In this section we will begin to develop these.

The approach has many similarities to ordinary algebra, where the variables all represent numbers. For example, $5 + 9 = 9 + 5$. This is an example of a general property of addition of numbers, which can be expressed as

$$a + b = b + a$$

where a and b might be any integers.

Boolean expressions also obey certain algebraic rules that enable us to manipulate them. The algebra of Boolean expressions has many similarities to the algebra of numbers — but also has some significant differences, so it is not helpful to take the analogy too far. However, one important rule in Boolean algebra is that brackets take precedence, just as they do in ordinary algebra. For example, to evaluate the expression $p \wedge (r \vee s)$, evaluate the $(r \vee s)$ part first.

Aside

There are software tools that implement these rules. Such tools, called theorem provers, can be used to prove theorems in logic. We include below the abstract of a paper — just to illustrate that such tools are used in industry.

Abstract. We examine the challenges presented by large-scale formal verification of industrial-size circuits, based on our experiences in verifying the class of all micro-operations executing on the floating-point division and square root unit of the Intel IA-32 Pentium 4 microprocessor. The verification methodology is based on combining human-guided mechanised theorem-proving with low-level steps verified by fully automated model-checking. A key observation in the work is the need to explicitly address the issues of proof design and proof engineering, i.e. the process of creating proofs and the craft of structuring and formulating them, as concerns in their own right.

Proof Engineering in the Large: Formal Verification of Pentium 4 Floating-Point Divider by Roope Kaivola and Katherine Kohatsu of the Intel Corporation. CHARME 2001, LCNS 2144, pp. 196–211.

End of Aside

3.1 Rules of Boolean algebra

Recall that, in Unit 9, we called two Boolean expressions **equivalent** if they always have the same truth values whatever the truth values of the atoms in the expressions. We use the symbol $\equiv$ to show that two Boolean expressions are equivalent; for example, we showed in Unit 9 that $\neg(a \vee b) \equiv \neg a \wedge \neg b$. The algebra of Boolean expressions is based on equivalences.

In this subsection we shall present a large number of rules that can be used to manipulate Boolean expressions. Each rule that we use is an equivalence. For reference, we give each of these rules a name. We shall not give justifications here for all of the rules. Some were covered in Unit 9. All the rules that we give can be justified by drawing up truth tables for each side of the equivalence and showing that they are identical.

In each of the results that we state, s , t and r are Boolean variables. We start by giving some results involving the connectives $\wedge$ and $\vee$.

Commutativity

$$\begin{aligned}s \vee t &\equiv t \vee s \\ s \wedge t &\equiv t \wedge s\end{aligned}$$

See Unit 9, Subsection 1.2.

Activity 1.3 of Unit 9.

Terms such as “commutativity” are used more generally, but that need not concern us here.

There is an algebra of sets that is closely related to Boolean algebra.

The results for sets corresponding to associativity are that the set equalities below are always **true**.

$$\begin{aligned}A \cup (B \cup C) &== (A \cup B) \cup C \\ A \cap (B \cap C) &== (A \cap B) \cap C.\end{aligned}$$

In general, the corresponding rule for sets is obtained by replacing $\vee$ by $\cup$ and $\wedge$ by $\cap$.

Associativity

$$\begin{aligned}s \vee (t \vee r) &\equiv (s \vee t) \vee r \\ s \wedge (t \wedge r) &\equiv (s \wedge t) \wedge r\end{aligned}$$

Distributivity

$$\begin{aligned}s \vee (t \wedge r) &\equiv (s \vee t) \wedge (s \vee r) \\ s \wedge (t \vee r) &\equiv (s \wedge t) \vee (s \wedge r) \\ \text{or, alternatively,} \\ (s \wedge t) \vee r &\equiv (s \vee r) \wedge (t \vee r) \\ (s \vee t) \wedge r &\equiv (s \wedge r) \vee (t \wedge r)\end{aligned}$$

Note that we have given alternative versions of the two laws of distributivity. The second pair of equivalences there follow from the first pair using commutativity. We have included both versions for convenience.

It is useful to observe that associativity tells us that we may omit the brackets if an expression only involves one of the connectives $\vee$ or $\wedge$. That is, we can write $s \vee t \vee r$ without creating confusion, because whichever way we put in the brackets we get equivalent expressions. If we combine that with commutativity we can go further, and see that the order in which we write terms in such an expression is not important either.

The rules of distributivity show that we need to be more careful if an expression involves both $\wedge$ and $\vee$. For example, the expression $(s \vee t) \wedge r$ is not equivalent to $s \vee (t \wedge r)$. In such a case, the brackets are essential to

Corresponding results are used freely in the algebra of numbers.

provide an unambiguous meaning, since the expression $s \vee t \wedge r$ cannot be interpreted. If an expression involves more than one connective, then careful attention is needed to make sure that any algebraic arrangement is valid.

In each of the rules, s , t and r can be single atoms, or they can be Boolean expressions. For example, suppose that a , b and c are atoms. Then the first Commutativity rule can be applied to the Boolean expressions $\neg a$ and $b \wedge c$, and shows that $\neg a \vee (b \wedge c) \equiv (b \wedge c) \vee \neg a$.

We have not given a rule of precedence between the connectives $\vee$ and $\wedge$.

Activity 3.1

Use algebra to show that $(a \vee b) \wedge (c \vee d) \equiv (a \wedge c) \vee (b \wedge c) \vee (a \wedge d) \vee (b \wedge d)$.

Discussion To establish this equivalence by algebra, we shall start with the left-hand side and use the rules to change it to the right-hand side. The first step uses distributivity to expand the brackets.

Of course, any such equivalence can be established by comparing the truth tables for the two expressions.

$$(a \vee b) \wedge (c \vee d) \equiv \underbrace{((a \vee b) \wedge c)}_{\text{first term}} \vee \underbrace{((a \vee b) \wedge d)}_{\text{second term}}$$

The right-hand side has now two terms, as indicated above. We can use distributivity again to expand the brackets in each term as follows.

$$\begin{aligned} (a \vee b) \wedge c &\equiv (a \wedge c) \vee (b \wedge c) \\ (a \vee b) \wedge d &\equiv (a \wedge d) \vee (b \wedge d) \end{aligned}$$

We can then substitute these expansions back in the right-hand side.

$$(a \vee b) \wedge (c \vee d) \equiv (a \wedge c) \vee (b \wedge c) \vee (a \wedge d) \vee (b \wedge d)$$

Here, associativity enables us to remove surplus brackets.

We usually write a manipulation such as this as a sequence of steps as follows.

$$\begin{aligned} (a \vee b) \wedge (c \vee d) &\equiv ((a \vee b) \wedge c) \vee ((a \vee b) \wedge d) \text{ (distributivity)} \\ &\equiv ((a \wedge c) \vee (b \wedge c)) \vee ((a \wedge d) \vee (b \wedge d)) \text{ (distributivity)} \\ &\equiv (a \wedge c) \vee (b \wedge c) \vee (a \wedge d) \vee (b \wedge d) \end{aligned}$$

We will not normally expect this level of detail when manipulating Boolean expressions algebraically. Usually, you do not need to explicitly state that you have used any of these first three rules in your work. If you are familiar with ordinary algebra you might like to compare Activity 3.1 with an algebraic manipulation such as $(a + b) \times (c + d) = a \times c + b \times c + a \times d + b \times d$, where a, b, c and d are numbers.

The remaining rules that we introduce do not have corresponding rules in the ordinary algebra of numbers.

Idempotency

$$\begin{aligned} s \vee s &\equiv s \\ s \wedge s &\equiv s \end{aligned}$$

The name “idempotency” comes from the Greek word “idios” for same. Thus the law is saying that the two terms taken together have the same effect (potency) as one.

In the next rule, and in subsequent text, we use (for brevity) T and F for the Boolean values **true** and **false**.

Absorption

$$\begin{aligned}s \vee T &\equiv T \\ s \vee F &\equiv s \\ s \wedge T &\equiv s \\ s \wedge F &\equiv F\end{aligned}$$

The name “absorption” indicates that in each equivalence one of the terms has been “absorbed”.

Activity 3.2

Show that $(a \vee b) \wedge a \equiv a \vee (a \wedge b)$.

Discussion We rearrange the expression on the left-hand side step-by-step, until we obtain the expression on the right-hand side.

$$\begin{aligned}(a \vee b) \wedge a &\equiv (a \wedge a) \vee (b \wedge a) && \text{Expanding the brackets may show a simplification.} \\ &\equiv a \vee (b \wedge a) \text{ (idempotency)} && \text{Ah yes! } a \wedge a \text{ collapses to } a. \\ &\equiv a \vee (a \wedge b) \text{ (commutativity)} && \text{One swap and we're there!}\end{aligned}$$

We often use the phrase “expanding the brackets” to indicate the use of distributivity.

Activity 3.3

Show that $(a \vee T) \vee (b \wedge c \wedge d) \equiv T$.

Discussion To show that the expression on the left-hand side is always **true**, we can use the first equivalence in absorption twice.

$$\begin{aligned}(a \vee T) \vee (b \wedge c \wedge d) &\equiv T \vee (b \wedge c \wedge d) \text{ (absorption)} \\ &\equiv (b \wedge c \wedge d) \vee T \text{ (commutativity)} \\ &\equiv T \text{ (absorption with } s = b \wedge c \wedge d)\end{aligned}$$

Notice how in the above discussion we regarded $(b \wedge c \wedge d)$ as a single term when applying the rules of commutativity and absorption. In the results we have given, s, t and r are Boolean variables. This means that they can be single atoms or they can be Boolean expressions.

The next two results concern expressions involving negation.

Negation

$$\begin{aligned}s \vee \neg s &\equiv T \quad \text{(i)} \\ s \wedge \neg s &\equiv F \quad \text{(ii)} \\ \neg \neg s &\equiv s \quad \text{(iii)}\end{aligned}$$

An expression that is always **true**, no matter what the truth value of its constituent variables, is called a **tautology**. So, for example, $s \vee \neg s$ is a tautology.

De Morgan

$$\neg(s \vee t) \equiv \neg s \wedge \neg t \quad (\text{i})$$

$$\neg(s \wedge t) \equiv \neg s \vee \neg t \quad (\text{ii})$$

De Morgan's laws were mentioned in Unit 7, in the context of set theory. We noted briefly earlier that there is a correspondence between Boolean algebra and an algebra of sets. Just as $\vee$ corresponds to $\cup$ on sets, and $\wedge$ corresponds to $\cap$, negation corresponds to the relative complement with the predefined set. Thus, if A and B are sets drawn from a predefined set U , the results of set algebra corresponding to the De Morgan rules are as follows.

$$U - (A \cup B) == (U - A) \cap (U - B)$$

$$U - (A \cap B) == (U - A) \cup (U - B)$$

Incidentally, to obtain results in set algebra corresponding to rules involving T or F, replace T by the predefined set and F by the empty set. So, for example, negation(ii) corresponds to a result that $A \cap (U - A) == \{\}$.

Augustus De Morgan (1806–1871) was appointed as the first professor of mathematics at University College London in 1828. He published many articles in the *Penny Cyclopaedia* which was published by the Society for the Diffusion of Useful Knowledge. He was the first president of the London Mathematical Society and — yes — he did introduce De Morgan's laws.

Activity 3.4

Use truth tables to show that the equivalence in De Morgan(ii) does hold.

Discussion Truth tables for the two expressions are given below.

		(2)		(1)	
s	t	$\neg$	$(s \wedge t)$	$\neg s$	$\neg t$
T	T	F	T	F	F
T	F	T	F	T	T
F	T	T	F	T	T
F	F	T	F	F	F

		(1)	
s	t	$\neg s$	$\neg t$
T	T	F	F
T	F	T	T
F	T	T	T
F	F	F	F

Column (2) in the first table is identical to column (1) in the second, showing that the required equivalence does hold.

$$\neg(s \wedge t) \equiv \neg s \vee \neg t$$

Activity 3.5

- Show that $\neg(s \wedge \neg t) \equiv \neg s \vee t$.
- Show that $s \wedge (\neg(s \wedge \neg t)) \equiv s \wedge t$.

Discussion In each case we start by using De Morgan in the hope of a simplification, and then use negation(iii) to remove double negation.

$$\begin{aligned} \text{(a)} \quad \neg(s \wedge \neg t) &\equiv \neg s \vee \neg \neg t && \text{(De Morgan)} \\ &\equiv \neg s \vee t && \text{(negation(iii))} \end{aligned}$$

$$\begin{aligned} \text{(b)} \quad s \wedge (\neg(s \wedge \neg t)) &\equiv s \wedge (\neg s \vee t) && \text{(De Morgan and negation(iii))} \\ &\equiv (s \wedge \neg s) \vee (s \wedge t) && \text{(distributivity)} \\ &\equiv \mathbf{F} \vee (s \wedge t) && \text{(negation(ii))} \\ &\equiv s \wedge t && \text{(absorption)} \end{aligned}$$

Expanding the brackets in hope of a simplification

The next result shows that we can always rewrite expressions involving the connectives $\Rightarrow$ or $\Leftrightarrow$ as expressions without them.

These equivalences were established in Exercise 4.2 in Unit 9.

Rewriting $\Rightarrow$

$$\begin{aligned} s \Rightarrow t &\equiv \neg s \vee t && \text{(i)} \\ s \Leftrightarrow t &\equiv (s \Rightarrow t) \wedge (t \Rightarrow s) && \text{(ii)} \end{aligned}$$

Activity 3.6

Show that $s \Rightarrow (s \vee t)$ is a tautology.

Discussion We start by removing $\Rightarrow$ from the expression.

$$\begin{aligned} s \Rightarrow (s \vee t) &\equiv \neg s \vee (s \vee t) && \text{(rewriting } \Rightarrow \text{)} \\ &\equiv (\neg s \vee s) \vee t && \text{(associativity)} \\ &\equiv \mathbf{T} \vee t && \text{(negation(i))} \\ &\equiv \mathbf{T} && \text{(absorption)} \end{aligned}$$

Since $s \Rightarrow (s \vee t)$ is equivalent to **true**, this Boolean expression is a tautology.

Activity 3.7

Show that the propositions below are equivalent. (They are propositions 2.2 and 2.1 on page 16.)

$$\forall \text{word in Dict} [\neg(\text{FIRST}(\text{word}, 'a') \wedge \text{LAST}(\text{word}, 'z'))]$$

$$\forall \text{word in Dict} [\text{FIRST}(\text{word}, 'a') \Rightarrow (\neg \text{LAST}(\text{word}, 'z'))]$$

Discussion Note that the binding terms in the two expressions are the same. Consequently, we can show that these propositions are equivalent by showing that the expressions within the two scopes are equivalent. The expression in the scope of the quantifier in the first proposition has the form $\neg(s \wedge t)$ which is equivalent to $\neg s \vee \neg t$ using De Morgan's law. The rule for rewriting $\Rightarrow$ shows that this expression can be written as $s \Rightarrow \neg t$, which is the form of the expression in the scope of the quantifier in the second proposition. Hence the two propositions are equivalent.

3.2 Equivalent expressions with quantifiers

The idea of equivalence can be extended to expressions involving quantifiers. To establish what is a valid equivalence in this case is complicated by the presence of variables. We cannot draw up a truth table for each side of the equivalence. To be equivalent, two propositions involving quantifiers should have the same truth values for any suitable values of the variables — but since quantification can be over an infinite set, a truth table to establish this would involve checking infinitely many cases. This is not a feasible approach! We shall rely here on an informal approach.

We shall not attempt to give a comprehensive list of equivalences — we just mention a few that occur rather frequently. An important point to note here is that equivalent propositions will always have the same meaning, for any given interpretation of the predicates and sets that they contain.

The first rule we give here says that in an expression involving two quantifications, we can reverse the order of the quantifiers *so long as both quantifiers are $\forall$ or both are $\exists$* . It is equally important to note that we can *not* reverse the order of two quantifiers when one is $\forall$ and one is $\exists$ — see, for example, Activity 2.6. These equivalences relate to a predicate $p(x, y)$, where x is drawn from U , and y is drawn from a set V .

The sets U and V could be the same, of course.

Extended commutativity

$$\begin{aligned}\forall x \text{ in } U [\forall y \text{ in } V [p(x, y)]] &\equiv \forall y \text{ in } V [\forall x \text{ in } U [p(x, y)]] \\ \exists x \text{ in } U [\exists y \text{ in } V [p(x, y)]] &\equiv \exists y \text{ in } V [\exists x \text{ in } U [p(x, y)]]\end{aligned}$$

In the first of these equivalences each proposition is **true** if $p(x, y)$ is **true** for every value of x in U and every value of y in V . In the second, each proposition is **true** if we can find at least one value of x in U and one value of y in V such that $p(x, y)$ is **true**.

The next two equivalences can be seen as an extension of De Morgan's laws.

Extended De Morgan

$$\begin{aligned}\neg \forall x \text{ in } U [p(x)] &\equiv \exists x \text{ in } U [\neg p(x)] & \text{(i)} \\ \neg \exists x \text{ in } U [p(x)] &\equiv \forall x \text{ in } U [\neg p(x)] & \text{(ii)}\end{aligned}$$

Look at equivalence (i) above. Either of the propositions there will be **true** so long as there is any value of x in U for which $p(x)$ is **false**. The proposition on the left there can be interpreted as saying

It is not the case that $p(x)$ is **true** for every x in U ,

while the right-hand side says

There exists x in U , such that $p(x)$ is **false**.

Now look at equivalence (ii) in extended De Morgan. Either of those propositions will be **true** if $p(x)$ is **false** for every x in U . There, the proposition

on the left says

There does not exist x in U such that $p(x)$ is **true**,

while the right-hand side says

For every x in U , $p(x)$ is **false**.

Activity 3.8

Write the following propositions in natural language. The sets and predicates are as defined on page 15.

- (a) $\exists c \text{ in Char}[\exists d \text{ in Char}[FIRST(\text{"hello"}, c) \wedge LAST(\text{"hello"}, d)]]$
 (b) $\neg \forall c \text{ in Char}[FIRST(\text{"hello"}, c)]$

Discussion

- (a) This proposition can be written as

There exists a character that is the first character of “hello” and a character that is the last character of “hello”.

It is clear that this means the same thing as

There exists a character that is the last character of “hello” and a character that is the first character of “hello”.

The order of the quantifiers does not affect the meaning.

- (b) This proposition can be interpreted as

It is not true that all characters are the first character in “hello”.

This could also be expressed as

There exists a character that is not the first character of “hello”.

In symbols that would more naturally be represented as below.

$\exists c \text{ in Char}[\neg FIRST(\text{"hello"}, c)]$

The extended De Morgan rule shows this proposition to be equivalent to that given in the question.

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Manipulate simple Boolean expressions using various rules, including
 - the commutative, associative and distributive rules;
 - the idempotent and absorption rules;
 - the rules about negatives;
 - De Morgan’s laws;
 - rules about implication.
- ☐ Recognise certain equivalent propositions involving quantifiers.

Exercises on Section 3

Exercise 3.1

Show each of (a)–(c), using the rules of Boolean algebra given in this section.

- (a) $s \wedge (t \vee \neg s) \equiv s \wedge t$.
- (b) $\neg(s \vee t) \vee t \equiv \neg s \vee t$.
- (c) $(s \wedge t) \Rightarrow t$ is a tautology.

Exercise 3.2

The predicates used here have the interpretations given on page 15. Express each proposition below in natural language. Are any of these pairs of propositions equivalent? Give the truth value of each proposition where *Dict* is the particular dictionary given on page 15.

- (a) $\forall \text{word in Dict} [\exists c \text{ in Char} [\text{SECOND}(\text{word}, c)]]$
 $\forall c \text{ in Char} [\exists \text{word in Dict} [\text{SECOND}(\text{word}, c)]]$
- (b) $\neg \forall \text{word in Dict} [\exists c \text{ in Char} [\text{SECOND}(\text{word}, c)]]$
 $\exists \text{word in Dict} [\neg \exists c \text{ in Char} [\text{SECOND}(\text{word}, c)]]$
- (c) $\forall \text{word in Dict} [\forall c \text{ in Char} [\text{SECOND}(\text{word}, c)]]$
 $\forall c \text{ in Char} [\forall \text{word in Dict} [\text{SECOND}(\text{word}, c)]]$

Exercise 3.3

For predicates p and q , use equivalences given in this section to show that:

$$\neg \forall x \text{ in Int} [p(x) \Rightarrow q(x)] \equiv \exists x \text{ in Int} [p(x) \wedge \neg q(x)].$$

4 Rules of deduction

In this section we shall consider logical arguments, and see how acceptable steps in such an argument can be expressed using symbols. To recognise valid forms of argument, we look for Boolean expressions of the form $s \Rightarrow t$ that are tautologies. Recall that a tautology is a Boolean expression that is **true** whatever the truth value of its atoms. If a Boolean expression of the form $s \Rightarrow t$ is a tautology, then we know that we cannot have a situation where s is **true** and t is **false**. Consequently, if we know that s is **true** then we can deduce that t must also be **true**. In most forms of argument, a deductive step will be based on more than one thing we already know to be **true**. So s will typically be a number of Boolean expressions combined using $\wedge$.

As an example, consider the following argument. In this, the first two statements are things we already know or believe to be true, while the third line is a deduction that we make on the basis of the first two lines.

If this animal is a cow then it can moo.

This animal cannot moo.

Hence this animal is not a cow.

To express this argument symbolically, first choose some simple propositions.

c : This animal is a cow.

m : This animal can moo.

The first two lines of argument then consist of the composite propositions:

$c \Rightarrow m$,

$\neg m$.

The argument claims that $\neg c$ follows from these.

We can express the argument symbolically using implication, as:

$((c \Rightarrow m) \wedge \neg m) \Rightarrow \neg c$.

To show that we have a valid form of argument, we would like to show that this expression is a tautology.

We have two ways in which we can show that a given Boolean expression is a tautology. One is to use the sort of algebraic arguments we considered in Section 3. Alternatively, we can use a truth table, and show that the expression is **true** in every line. In this case, the expression involves only two atoms, but is rather complicated. In such a case, a truth table provides the more convenient approach.

That is, if we know the propositions $c \Rightarrow m$ and $\neg m$ to be **true**, then we can deduce that $\neg c$ is **true**.

Activity 4.1

Draw up a truth table to show that $((c \Rightarrow m) \wedge \neg m) \Rightarrow \neg c$ is a tautology.

Discussion A truth table is given below. Note that we do indeed have **true** in each row of the table.

c	m	$((c \Rightarrow m) \wedge \neg m) \Rightarrow \neg c$	giving	c	m	$((c \Rightarrow m) \wedge \neg m) \Rightarrow \neg c$
T	T	T		T	T	T
T	F	F		T	F	T
F	T	T		F	T	T
F	F	T		F	F	T

The fact that $((c \Rightarrow m) \wedge \neg m) \Rightarrow \neg c$ is a tautology has nothing to do with the interpretation given earlier to the simple propositions m and c . Such a tautology corresponds to a form of argument that is valid in general, whatever the meaning of the simple propositions that it contains. Our purpose in this section is to develop a “library” of forms of argument that are valid in general. Each such form of argument is supported by a tautology.

We first introduce some general terminology. In formal logic, an **argument** is a claim that if all of a set of propositions, say $s_1, s_2, \dots, s_n$, are **true** then we can deduce that another proposition, say t , is **true**. Such an argument is a **valid argument** if $(s_1 \wedge s_2 \wedge \dots \wedge s_n) \Rightarrow t$ is a tautology.

In an argument corresponding to an expression of the form

$$(s_1 \wedge s_2 \wedge \dots \wedge s_n) \Rightarrow t,$$

we call $s_1, s_2, \dots, s_n$ the **premises** of the argument and t the **conclusion** of the argument. If the expression is a tautology, so that we have a valid argument, t is called a **semantic consequence** of $s_1 \wedge s_2 \wedge \dots \wedge s_n$.

For example, in the argument discussed above, the premises are $c \Rightarrow m$ and $\neg m$ and the conclusion is $\neg c$. As the argument is valid, $\neg c$ is a semantic

An argument with two premises and one conclusion is known as a **syllogism**.

consequence of $c \Rightarrow m$ and $\neg m$. We know this argument to be valid because we showed in Activity 4.1 that $((c \Rightarrow m) \wedge \neg m) \Rightarrow \neg c$ is a tautology.

Activity 4.2

For each of the following arguments, give suitable simple propositions, and use these to write down the premises and conclusion of the argument in symbolic form. Then use truth tables to determine whether the form of argument is valid.

- (a) We cannot play cricket if it is raining.
It is not raining.
So we can play cricket.
- (b) If my ISP server is busy then I cannot send an email.
I can send an email so my ISP server is not busy.

Discussion

- (a) Each assertion is formed from the same two simple propositions.

c : We can play cricket.

r : It is raining.

The premises and conclusion are clearly separated here: the first two assertions are the premises of the argument, and the third is the conclusion. The first premise is $r \Rightarrow \neg c$ ("If it is raining then we cannot play cricket."). The second premise is $\neg r$. The conclusion is c .

So the form of the argument is $((r \Rightarrow \neg c) \wedge \neg r) \Rightarrow c$.

The truth table for this Boolean expression is given below.

c	r	(1)	(2)	(3)	
c	r	$((r \Rightarrow \neg c) \wedge \neg r)$	$\Rightarrow$	c	
T	T	F	F	T	T
T	F	F	T	T	T
F	T	T	F	T	F
F	F	T	T	F	F

giving

c	r	$((r \Rightarrow \neg c) \wedge \neg r) \Rightarrow c$
T	T	T
T	F	T
F	T	T
F	F	F

The last line of the truth table shows that this is not a tautology. Consequently, this argument is not valid. This illustrates how easy it is to get an argument wrong. It could be that the weather was fine, but we did not have enough players available to form a team, so we could not play cricket.

- (b) The two assertions can again be formed using two simple propositions.

s : My ISP server is busy.

e : I can send email.

This time, the premises and conclusion are not clearly separated. The first sentence corresponds to the composite proposition $s \Rightarrow \neg e$. This forms one premise of the argument. The second sentence combines a second premise and the conclusion of the argument. The conclusion forms the latter part of the second sentence, and is $\neg s$. The first part of the second sentence contains a second premise, namely e .

The form of this argument is $((s \Rightarrow \neg e) \wedge e) \Rightarrow \neg s$.

A truth table for this Boolean expression is given below. This time, the last column has all values **true**, so we have a tautology. The argument given here is valid.

s	e	(1)	(2)	(3)	
$((s \Rightarrow \neg e) \wedge e) \Rightarrow \neg s$					
T	T	F	F	T	F
T	F	T	F	F	F
F	T	T	T	T	T
F	F	T	F	T	T

giving

s	e	$((s \Rightarrow \neg e) \wedge e) \Rightarrow \neg s$
T	T	T
T	F	T
F	T	T
F	F	T

Checking that arguments are valid is not just an academic exercise, but one of the skills needed when verifying that a piece of software does what it claims. Some software packages are **safety critical** in that they are used to control systems on which lives may depend. Systems that control railway signalling or automatic pilots are examples of such software.

For example, consider the control of a rail signalling system. In practice this will be very complicated, with factors such as positions of trains and various points settings all being taken into account when deciding whether a train can move. As a simple example, we might want to use the argument below.

If traffic is allowed to flow then the light is green.

The light is not green.

Hence traffic is not allowed to flow.

We would need to be sure that this is a valid argument before using it.

The aim of this section is to give you a flavour of how we can work with such arguments. There will be two stages to go through. The first is to extract simple propositions, give them identifiers, and express the argument symbolically using these. The second is to check the validity of the argument.

The second stage is shortened by using rules of inference. These are forms of argument — expressed symbolically — that are known to be valid. In other words they are essentially short cuts that we can use rather than repeatedly checking the validity of the same argument in different contexts.

In the argument given above, we can identify the simple propositions as

g : The light is green.

t : The traffic is allowed to flow.

Then the premises of the argument are:

$t \Rightarrow g$

$\neg g$.

The conclusion is $\neg t$. This argument has exactly the same form as did the first argument we gave in this section. We can use the tautology established in Activity 4.1 to conclude that this argument is valid as well. (Replace c by t and m by g . We have a tautology whatever symbols are used for the atoms, so long as the form of the Boolean expression is the same.)

So far, we have looked at the validity of arguments on an *ad hoc* basis. Having identified the premises and conclusion of an argument, we have used a truth table to test whether or not the corresponding Boolean expression is a tautology. However, as just noted, different arguments may have the same underlying form. We now move on to exploit this in a systematic way.

In the abstract on page 26, the word “mechanised” is used to indicate the use of a theorem prover. A theorem prover will have the rules of inference that we shall develop built in to the system. Some theorem provers in use today are known as HOL, EVES, PVS, and Isabelle.

4.1 Rules of inference

We now start to develop a library of “templates”, each showing a form of argument that is valid — because there is a corresponding tautology. We then do not need to draw up a truth table each time to check whether we have a tautology — we just note that the argument is of the form given in one of our templates. Each such valid form of argument that we identify is called a **rule of inference**. These rules identify some common patterns in valid arguments. We will introduce these rules of inference gradually, but for easy reference we have gathered them together on page 43.

By a **proof** of an argument we shall mean a finite sequence of propositions, each of which is either a premise or can be deduced from the previous propositions, ending with the conclusion. We can develop proofs using the rules of Boolean algebra given in Section 3. But the rules of inference developed here provide a way of taking short cuts in developing proofs.

To express rules of inference, we now introduce a new notation. We write:

$$\frac{s_1; s_2; \dots; s_n}{t}$$

to mean that if the propositions $s_1; s_2; \dots; s_n$ are **true** then we can deduce that the proposition t is **true**. We could replace each semi-colon by the symbol $\wedge$, but some of the expressions s_i may themselves include $\wedge$. This notation makes the premises of the argument clear. The propositions above the line are the premises and the proposition below the line is the conclusion.

Each of the inference rules that we give is a form of argument that is always valid. The first that we will introduce is referred to as the implication rule.

$$\text{implication rule} \quad \frac{s \Rightarrow t; s}{t}$$

If we know s
and we know that s implies t
then we can deduce t .

Activity 4.3

Consider the following argument.

If Fido is a dog then Fido barks at postmen.

Fido is a dog.

Hence Fido barks at postmen.

Decide on some simple propositions that are used in the argument and allocate symbols to these propositions. Write down the premises and conclusion of the argument in terms of these symbols. Show that the argument is valid.

Discussion We first allocate symbols to simple propositions.

dog: Fido is a dog.

post: Fido barks at postmen.

Then the premises of the given argument are $dog \Rightarrow post$ and dog and the conclusion is $post$.

Now if, in the implication rule, we replace s by dog and t by $post$, we obtain

$$\frac{dog \Rightarrow post; dog}{post}$$

Above the line here we have the premises of the given argument, so we can deduce that $post$ is a valid conclusion. Thus the given argument is valid. (If the premises are **true**, then “Fido barks at postmen” is **true**.)

Use of such names for propositions, rather than single symbols, can help one to remember the interpretations.

The next rule of inference that we shall use is the contrapositive rule.

$$\text{contrapositive rule} \quad \frac{s \Rightarrow t; \neg t}{\neg s}$$

This is not quite as easy to see as the implication rule. Incidentally, care is needed when one or more of the premises is a negation. We need to be clear that $\neg t$ is **true** means that we know that t is **false**, not just that we do not know that t is **true**.

Each inference rule that we will introduce can be justified by showing that the appropriate Boolean expression is a tautology. For example, the implication rule can be shown to be valid using the truth table below.

s	t	(1)	(2)	(3)	
s	t	$((s \Rightarrow t) \wedge s) \Rightarrow t$	$\wedge$	s	t
T	T	T	T	T	T
T	F	F	F	T	F
F	T	T	F	F	T
F	F	T	F	F	F

giving

s	t	$((s \Rightarrow t) \wedge s) \Rightarrow t$
T	T	T
T	F	F
F	T	T
F	F	T

We will not show the truth table for all the rules of inference, since the main aim of this discussion is to learn how to use the rules.

If we know that s implies t and we know that t is **false** then we can deduce that s is **false**.

Confusion is possible if we do not have complete information about the domain of interest.

The tautology justifying the contrapositive rule was established in Activity 4.1.

Activity 4.4

Consider the following argument.

If this animal is a donkey then it has long ears.

This animal has not got long ears.

Hence this animal is not a donkey.

Decide on some simple propositions that are used in the argument and allocate symbols to these propositions. Write down the premises and conclusion of the argument in terms of these symbols. Show that the argument is valid.

Discussion We first allocate symbols to simple propositions.

donkey: This animal is a donkey.

ears: This animal has long ears.

Then the premises of the given argument can be written as $donkey \Rightarrow ears$ and $\neg ears$, and the conclusion as $\neg donkey$.

In the contrapositive rule, if we replace s by *donkey* and t by *ears*, we obtain

$$\frac{donkey \Rightarrow ears; \neg ears}{\neg donkey}$$

Above the line are the premises of the given argument, so we can deduce that $\neg donkey$ is a valid conclusion. Thus the argument is valid. (If the premises are **true**, then we can deduce that “This animal is not a donkey.” is **true**.)

Each example above involved a single rule of inference. We now develop an approach to deal with more complex situations.

Activity 4.5

The argument below has three premises and a conclusion. Use the rules of inference to show that the argument is valid.

Premises

- (1) If the music is loud at night then the neighbours cannot sleep.
- (2) If the neighbours cannot sleep then they are annoyed.
- (3) The music is loud at night.

Conclusion

The neighbours are annoyed.

Discussion We first allocate symbols to simple propositions.

music: The music is loud at night.

nosleep: The neighbours cannot sleep.

annoyed: The neighbours are annoyed.

We now write each of the premises symbolically using these.

- $$\begin{array}{ll} \textit{music} \Rightarrow \textit{nosleep} & (1) \\ \textit{nosleep} \Rightarrow \textit{annoyed} & (2) \\ \textit{music} & (3) \end{array}$$

The conclusion is *annoyed*.

We will now show that the given argument is valid. The lay-out we adopt is to list propositions which are **true**. For each proposition that we write down, we justify its inclusion either by indicating that it is a premise or giving the rule of inference that allows it to be concluded.

- $$\begin{array}{ll} \textit{music} \Rightarrow \textit{nosleep} & (1) \text{ premise} \\ \textit{nosleep} \Rightarrow \textit{annoyed} & (2) \text{ premise} \\ \textit{music} & (3) \text{ premise} \\ \textit{nosleep} & (4) \text{ implication rule with 1, 3} \\ \textit{annoyed} & (5) \text{ implication rule with 2, 4} \end{array}$$

We can summarise this argument as below, writing the premises above the line and the conclusion below it.

$$\frac{\textit{music} \Rightarrow \textit{nosleep}; \textit{nosleep} \Rightarrow \textit{annoyed}; \textit{music}}{\textit{annoyed}}$$

In this proof, how did we decide which rule to use and when? Our approach was to write down all the premises first. We then see if any combination of some or all of the premises gives us the top line of any of the rules of inference. We then add the conclusion of that rule of inference to our list of propositions known to be **true**, and repeat the process. If the conclusion of a rule of inference has already been obtained, then do not repeat it. It is, of course, important to keep the required conclusion in mind.

This becomes easier to see when you have had a little practice.

The next rule that we introduce is easy to understand and can be useful in removing irrelevant facts from an argument.

$$\text{simplication rule} \quad \frac{s; t}{s}.$$

This rule says that if we know that both *s* and *t* are true then we know that *s* is true.

Activity 4.6

Show that the following is a valid argument.

Premises

- (1) If I am hungry then I will have lunch now.
- (2) I will neither have lunch nor will I have a cup of tea.

Conclusion

I am not hungry.

Discussion We first allocate symbols to simple propositions.

hungry: I am hungry.

lunch: I will have lunch.

tea: I will have a cup of tea.

The premises are expressed formally below. The second premise here needs care. Natural language can be imprecise when using negatives. The wording “neither ... nor ...” is most naturally represented as $\neg(lunch \vee tea)$ so we have used this form. However, this is equivalent to $\neg lunch \wedge \neg tea$, and you may have chosen that as a premise.

$hungry \Rightarrow lunch$ (1)

$\neg(lunch \vee tea)$ (2)

The conclusion is $\neg hungry$.

We now show that the given argument is valid.

- $hungry \Rightarrow lunch$ (1) premise
- $\neg(lunch \vee tea)$ (2) premise
- $\neg lunch \wedge \neg tea$ (3) equivalence to 2 by De Morgan’s law
- $\neg lunch$ (4) simplification of 3
- $\neg hungry$ (5) contrapositive rule with 1, 4

An important point is illustrated here: we can, if we wish, use the rules of Boolean algebra to rewrite an expression in a proof.

Thus we have shown that $\frac{hungry \Rightarrow lunch; \neg(lunch \vee tea)}{\neg hungry}$.

The next rule of inference is quite subtle.

$$\text{combining } \vee, \Rightarrow \quad \frac{s \vee t; s \Rightarrow r}{r \vee t}$$

If we know that either s or t is **true** and that s implies r then we know that either r or t is **true**.

Activity 4.7

Show that the following is a valid argument.

Premises

- (1) Either the post has not come or I have a letter.
- (2) If the post has not come then the dog is outside.
- (3) If I have a letter then I must write a letter.

Conclusion

Either the dog is outside or I must write a letter.

Discussion We first choose simple propositions. We could choose “The post has come.” as a simple proposition. However, this would need to be negated every time that it is used so we have used the negative assertion as our simple proposition. This makes the symbolic argument easier to read.

nopost: The post has not come.

letter: I have a letter.

dog: The dog is outside.

write: I must write a letter.

We now write each of the premises symbolically using these.

nopost $\vee$ *letter* (1)

nopost $\Rightarrow$ *dog* (2)

letter $\Rightarrow$ *write* (3)

The conclusion is *dog* $\vee$ *write*.

Our proof that this argument is valid is given below.

nopost $\vee$ *letter* (1) premise

nopost $\Rightarrow$ *dog* (2) premise

letter $\Rightarrow$ *write* (3) premise

dog $\vee$ *letter* (4) combining $\vee, \Rightarrow, 1, 2$

dog $\vee$ *write* (5) combining $\vee, \Rightarrow, 3, 4$

Hence
$$\frac{\textit{nopost} \vee \textit{letter}; \textit{nopost} \Rightarrow \textit{dog}; \textit{letter} \Rightarrow \textit{write}}{\textit{dog} \vee \textit{write}}.$$

You may have arrived at the conclusion by a different route to that set out by us. This is fine; it is the validity of the proof that counts.

In the discussion of Activity 4.6, we used a rule of Boolean algebra to replace a compound proposition by one that is equivalent. It may seem self-evident that this is a valid step in a proof, but it is useful to have a formal justification. This can be provided by the following rule

$$\text{equivalence rule} \quad \frac{s \Leftrightarrow t; s}{t}$$

This rule says that if we know that *s* is equivalent to *t* and we know that *s* is true we may deduce that *t* is true.

Each rule of Boolean algebra in Section 3 is based on an equivalence. We have $s \equiv t$ if the truth values of *s* and *t* are always the same (whatever the truth value of their atoms). Now $s \Leftrightarrow t$ is **true** if *s* and *t* have the same truth values. So if $s \equiv t$ then $s \Leftrightarrow t$ will always be **true**; that is, $s \Leftrightarrow t$ is a tautology. In any argument we would be justified in adding a tautology as an extra premise if need be.

Result 4.1, below, lists all the rules of inference that we have used above. It also includes some further rules that can be useful. These are mostly fairly natural. The addition rule says that if we know *s* is **true** then we can deduce that $s \vee t$ is **true**. Conjunction says that if we know that each of *s* and *t* is **true** as a separate proposition, then we know that the combined proposition $s \wedge t$ is **true**. Disjunction says that if we know that $s \vee t$ is **true**, and that *s* is **false**, then we can deduce that *t* is **true**. Transitivity says that we can combine two implications of the form $s \Rightarrow t$ and $t \Rightarrow r$ to deduce a single implication $s \Rightarrow r$.

Incidentally, it is not possible to show that an argument is invalid using rules of inference. (Just because one cannot see how to obtain some conclusion does not mean that it is impossible to obtain it!) To show that an argument is invalid, we need to give values for the variables for which the premises are all **true** and the conclusion is **false**. If such values exist, they can be found by drawing up a truth table as in Activity 4.2(a).

Result 4.1 Rules of inference for proofs

Implication	$\frac{s; s \Rightarrow t}{t}$
Contrapositive	$\frac{\neg t; s \Rightarrow t}{\neg s}$
Simplification	$\frac{s \wedge t}{s}$
Combining $\vee, \Rightarrow$	$\frac{s \vee t; s \Rightarrow r}{r \vee t}$
Equivalence	$\frac{s \Leftrightarrow t; s}{t} \text{ and } \frac{s \Leftrightarrow t; t}{s}$
Addition	$\frac{s}{s \vee t}$
Conjunction	$\frac{s; t}{s \wedge t}$
Disjunction	$\frac{\neg s; s \vee t}{t}$
Transitivity	$\frac{s \Rightarrow t; t \Rightarrow r}{s \Rightarrow r}$

The implication rule is also known by the Latin term, **modus ponens** and the contrapositive rule is known as **modus tollens**.

The contrapositive rule may also be referred to as contraposition.

Activity 4.8

Show that the following argument is valid.

Premises

If the bird nests under our eaves then it is a swift or a sparrow.

If the bird is a swift then it sleeps while flying.

The bird nests under our eaves.

The bird does not sleep while flying.

Conclusion

The bird is a sparrow.

Discussion We use the following simple propositions.

swift: The bird is a swift.

nests: The bird nests under our eaves.

sparrow: The bird is a sparrow.

sleeps: The bird sleeps while flying.

The premises are as follows.

$$nests \Rightarrow (swift \vee sparrow) \quad (1)$$

$$swift \Rightarrow sleeps \quad (2)$$

$$nests \quad (3)$$

$$\neg sleeps \quad (4)$$

We want a proof leading to the conclusion *sparrow*.

$nests \Rightarrow (swift \vee sparrow)$	(1) premise
$swift \Rightarrow sleeps$	(2) premise
$nests$	(3) premise
$\neg sleeps$	(4) premise
$swift \vee sparrow$	(5) implication 1,3
$\neg swift$	(6) contrapositive 2,4
$sparrow$	(7) disjunction 5,6
Hence	$\frac{nests \Rightarrow (swift \vee sparrow); swift \Rightarrow sleeps; nests; \neg sleeps}{sparrow}$

Activity 4.9

Show that the following argument is valid.

Premises

If I go for a walk in the mud then my boots need cleaning.

If I go for a walk in November then I go for a walk in the mud.

I go for a walk in November.

Conclusion

My boots need cleaning.

Discussion We use the following symbols for simple propositions.

mud: I go for a walk in the mud.

boots: My boots need cleaning.

nov: I go for a walk in November.

The premises are then

$mud \Rightarrow boots$	(1)
$nov \Rightarrow mud$	(2)
nov	(3)

We want a proof leading to the conclusion *boots*.

$mud \Rightarrow boots$	(1) premise
$nov \Rightarrow mud$	(2) premise
nov	(3) premise
mud	(4) implication 2,3
$boots$	(5) implication 1,4
Hence	$\frac{mud \Rightarrow boots; nov \Rightarrow mud; nov}{boots}$

Alternatively, one could use the transitivity rule.

4.2 Contradiction

The next example illustrates a problem that can occur, particularly when there are several composite propositions in an argument.

EXAMPLE 4.1

Think for a moment about whether the following is a valid argument.

Premises

- (1) On Mondays I am late for work.
- (2) The boss is in a good mood or I have time for lunch.
- (3) It is Monday and I have no time for lunch.
- (4) If I am late for work, then the boss is not in a good mood.

Conclusion

The boss is not in a good mood.

So far, there is no agreement between the members of the course team about this argument. Anton looks at it and argues that from (3), it is Monday. Then, from (1), I am late for work. Consequently, from (4) the boss is not in a good mood. However, Barbara argues that we know from (3) that I have no time for lunch. Consequently, from (2), the boss is in a good mood.

Who is right?

The answer is somewhat confusing, since both strands of reasoning follow correctly from the premises. However, “The boss is in a good mood.” cannot be both true and false at the same time. The problem here is that the original premises themselves contain this contradiction. In other words, they are not consistent. Formally, we define consistency as follows.

A set of propositions $p_1, p_2, \dots, p_n$ is called **consistent** if it is possible for $p_1 \wedge p_2 \wedge \dots \wedge p_n$ to be **true**.

A Boolean expression is a **contradiction** if it is equivalent to **false**. So the set of propositions $p_1, p_2, \dots, p_n$ is not consistent if $p_1 \wedge p_2 \wedge \dots \wedge p_n$ is a contradiction.

A contradiction is the negation of a tautology.

How can we tell that the premises in Example 4.1 are contradictory? It is not obvious when the premises are stated in English. To prove that they are not consistent, we first express the premises in symbolic language. We then give a proof that they lead to the conclusion **false**.

We can choose simple propositions as follows.

mon: It is Monday.

late: I am late for work.

lunch: I have time for lunch.

boss: The boss is in a good mood.

We can then write the premises symbolically as follows.

$$mon \Rightarrow late \quad (1)$$

$$boss \vee lunch \quad (2)$$

$$mon \wedge \neg lunch \quad (3)$$

$$late \Rightarrow \neg boss \quad (4)$$

We can give a formal argument that the premises taken together lead to the conclusion **false** as follows.

$mon \Rightarrow late$	(1) premise
$boss \vee lunch$	(2) premise
$mon \wedge \neg lunch$	(3) premise
$late \Rightarrow \neg boss$	(4) premise
mon	(5) simplification 3
$late$	(6) implication 1,5
$\neg boss$	(7) implication 4,6
$\neg lunch$	(8) simplification 3
$boss$	(9) disjunction 2,8
$boss \wedge \neg boss$	(10) conjunction 7,9
false	(11) equivalence to 10 by negation(ii)
Hence	$\frac{mon \Rightarrow late; boss \vee lunch; mon \wedge \neg lunch; late \Rightarrow \neg boss}{\text{false}}$

Aside

This discussion may have made you think that you should check that the premises are consistent before analysing an argument. In fact, if you are just showing an argument is valid, there is no need to do so. Both Anton and Barbara had valid arguments in Example 4.1. Remember that the proposition $s \Rightarrow t$ is **true** if s is **false**. This means that any argument $(s_1 \wedge s_2 \wedge \dots \wedge s_n) \Rightarrow t$, where the premises $s_1, s_2, \dots, s_n$ are contradictory, is valid. Of course, even with a valid argument, you should only deduce that the conclusion is **true** if you know that the premises are **true**!

You can show that a set of premises is consistent by giving truth values for the simple propositions which ensure that all the premises are **true**. For example in Activity 4.9, if all the simple propositions *mud*, *boots* and *nov* are **true**, then all the premises have the value **true**. This is enough to show that they are consistent.

End of Aside

Objectives for Section 4

After studying this section you should be able to do the following.

- ☐ Identify the premises and conclusion in an argument.
- ☐ Choose simple propositions and so express an argument in symbols.
- ☐ Show that a simple argument is valid using the rules of inference.
- ☐ Show that a simple argument is not valid using a truth table.
- ☐ Show that a set of premises is inconsistent.

Exercises on Section 4

Exercise 4.1

Express the following argument symbolically and hence show that it is valid.

If this animal is a cat then it does not bark.

This animal barks.

Hence this animal is not a cat.

Exercise 4.2

Give a truth table to show that disjunction is a valid rule of inference.

Exercise 4.3

Show that the following argument is not valid.

If this animal is a cat then it dislikes dogs.

This animal is not a cat.

Hence this animal likes dogs.

Exercise 4.4

Show that the following propositions are inconsistent.

I like this music.

If music is not beautiful then I do not like it.

If this music has not been recorded then it is not beautiful.

This music has not been recorded.

Summary of Unit 10

1. Logic is a tool in computing, and the aim of this unit is to increase your awareness of its uses. Predicates are an important component in the formal language of logic. The use of predicates in expressing conditional statements in code was illustrated in a small case study.
2. Quantifiers add to the variety of natural language assertions that can be expressed formally. There are two quantifiers: $\forall$, read as “for all” and $\exists$, read as “there exists”. Each quantifier binds a variable to a particular set in the domain of interest. Truth values for quantified propositions may be determined by prior knowledge of the domain of interest, but our usual approach here is to enumerate explicitly the sets in the domain of interest.
3. We gave a number of rules that can be used to manipulate Boolean expressions algebraically. Each such rule is based on an equivalence of compound Boolean expressions. Certain equivalences between propositions involving quantifiers were also given. When translating between formal expressions of propositions and their interpretation in natural language, it is helpful to be aware of such equivalences.
4. Rules of deduction can be used to check whether an argument is valid. The formal rules used here are summarised in Result 4.1.

Solutions to the exercises

Solution 1.1

$PREY$ (“robins”) and $PREDATOR$ (“sparrowhawks”) are both **true** but $BIGGER$ (“sparrowhawks”, “robins”) is **false**. So this proposition has truth value $(\mathbf{true} \wedge \mathbf{true}) \Rightarrow \mathbf{false}$, which gives **false**.

Note that we are only able to use the information given. In fact, sparrowhawks are bigger than robins. However, (“sparrowhawks”, “robins”) was not given as a member of the set *Bigger*.

Solution 1.2

We can put together ideas from Activities 1.6 and 1.7 to give an implementation of *MOVENOTOK*, as below. You may have expressed this in some other way, for example by using fewer **if** clauses and combining conditions using $\vee$.

```
function MOVENOTOK(t)
{ // t is of type TurtleBox
  var b in Bool
  b <-- false
  if (INBOX(t)) then
  { // code for t inside the box
    if ((t.getDirection()==0)/\ (t.getX()==t.getRight()-1)) then
      {b <-- true}
    if ((t.getDirection()==1)/\ (t.getY()==t.getTop()-1)) then
      {b <-- true}
    if ((t.getDirection()==2)/\ (t.getX()==t.getLeft()+1)) then
      {b <-- true}
    if ((t.getDirection()==3)/\ (t.getY()==t.getBottom()+1)) then
      {b <-- true}
  }
  else
  { // code for t outside the box
    if ((t.getDirection()==0)/\ (t.getX()==t.getLeft()-1)
      /\ (t.getBottom()<=t.getY())/ \ (t.getY()<=t.getTop())) then
      {b <-- true}
    if ((t.getDirection()==1)/\ (t.getY()==t.getBottom()-1)
      /\ (t.getLeft()<=t.getX())/ \ (t.getX()<=t.getRight())) then
      {b <-- true}
    if ((t.getDirection()==2)/\ (t.getX()==t.getRight()+1)
      /\ (t.getBottom()<=t.getY())/ \ (t.getY()<=t.getTop())) then
      {b <-- true}
    if ((t.getDirection()==3)/\ (t.getY()==t.getTop()+1)
      /\ (t.getLeft()<=t.getX())/ \ (t.getX()<=t.getRight())) then
      {b <-- true}
  }
  return b
}
```

Solution 2.1

(a) All birds are either seabirds or prey.

This proposition is **false**: for example, sparrowhawks are in *Birds*, but are neither seabirds nor prey.

(b) Ostriches are bigger than all the seabirds.

This proposition is **false**: for example, gannets are in the set *Seabirds*, but the set *Bigger* does not contain (“ostriches”, “gannets”).

(c) There is a bird that is both a seabird and a predator.

This proposition is **true**: for example, skuas are both seabirds and predators.

(d) There is a bird that is bigger than all the seabirds.

This proposition is **false**. For example, skuas are in *Seabirds* but do not appear in the set *Bigger* — so there is no bird that is bigger than skuas.

In an example such as this, do not rely on “common sense” when assigning truth values. Refer to the specific sets given. We have incomplete knowledge of this situation. In the real world, ostriches are bigger than all the birds in *Seabirds*, but this information is not recorded in the given set *Bigger*.

Solution 2.2

(a) $\forall \text{name in Birds} [SEABIRD(\text{name}) \Rightarrow BIGGER(\text{name}, \text{“goldcrests”})]$

(b) $\neg \exists \text{name in Birds} [PREDATOR(\text{name}) \wedge PREY(\text{name})]$

(c) $\exists \text{nameOne in Birds} [SEABIRD(\text{nameOne})] \wedge \exists \text{nameTwo in Birds} [PREDATOR(\text{nameTwo})]$

or

$\exists \text{nameOne in Birds} [\exists \text{nameTwo in Birds} [SEABIRD(\text{nameOne}) \wedge PREDATOR(\text{nameTwo})]]$

Solution 2.3

(a) The set is: $A(s) = \{i \text{ in Int} : (1 \leq i) \wedge (i \leq s.length())\}$.

(b) For the precondition we need to assert that the string s is not empty, and that there is a position in the string s where the character c occurs. We can do this as follows.

$$(s.length() > 0) \wedge \exists i \text{ in } A(s) [s.charAt(i) == c]$$

For the postcondition we need to say that the returned value represents a position in the string s , and that the character c occurs in s at the position whose index is the returned value. We can express this as below.

$$(FIRSTOCCUR(s, c) \in A(s)) \wedge (s.charAt(FIRSTOCCUR(s, c)) == c)$$

We also need to say that c does not occur in any position before the returned value.

$$\forall i \text{ in } A(s) [(i < FIRSTOCCUR(s, c)) \Rightarrow \neg (s.charAt(i) == c)]$$

We put this together to obtain the formal specification of the function as below.

function *FIRSTOCCUR*(s in mString, c in Char) **return** in Int

pre $(s.length() > 0) \wedge \exists i \text{ in } A(s) [s.charAt(i) == c]$.

post $(FIRSTOCCUR(s, c) \in A(s)) \wedge (s.charAt(FIRSTOCCUR(s, c)) == c) \wedge \forall i \text{ in } A(s) [(i < FIRSTOCCUR(s, c)) \Rightarrow \neg (s.charAt(i) == c)]$

Solution 3.1

(a) We first use distributivity to expand the brackets.

$$\begin{aligned} s \wedge (t \vee \neg s) &\equiv (s \wedge t) \vee (s \wedge \neg s) \text{ (distributivity)} \\ &\equiv (s \wedge t) \vee \text{F} \text{ (negation(ii))} \\ &\equiv s \wedge t \text{ (absorption)} \end{aligned}$$

(b) We first use De Morgan, then expand brackets in the hope of a simplification.

$$\begin{aligned} \neg(s \vee t) \vee t &\equiv (\neg s \wedge \neg t) \vee t \text{ (De Morgan)} \\ &\equiv (\neg s \vee t) \wedge (\neg t \vee t) \text{ (distributivity)} \\ &\equiv (\neg s \vee t) \wedge \text{T} \text{ (negation(i))} \\ &\equiv \neg s \vee t \text{ (absorption)} \end{aligned}$$

(c) We first use the rule for replacing $\Rightarrow$.

$$\begin{aligned}
 (s \wedge t) \Rightarrow t &\equiv \neg(s \wedge t) \vee t \text{ (replacing } \Rightarrow) \\
 &\equiv (\neg s \vee \neg t) \vee t \text{ (De Morgan(ii))} \\
 &\equiv \neg s \vee (\neg t \vee t) \text{ (associativity)} \\
 &\equiv \neg s \vee T \text{ (negation(i))} \\
 &\equiv T \text{ (absorption)}
 \end{aligned}$$

Since this proposition is always **true**, it is a tautology.

Solution 3.2

(a) Every word in the dictionary has a second character.

Every character is the second character of some word in the dictionary.

These propositions are not equivalent. The first is **true** for the given dictionary, as every word has at least two characters. The second is **false** as there is no word with 'k' as the second character (for example).

(b) It is not true that every word in the dictionary has a second character.

There is a word in the dictionary that does not have a second character.

These propositions are equivalent and are both **false** for the given dictionary.

(c) Every word in the dictionary has every character as its second character.

Every character is the second character of every word in the dictionary.

These propositions are equivalent and are clearly **false**.

Solution 3.3

We first use the extended De Morgan law, then use Boolean algebra to get the scope into the form on the right-hand side.

$$\begin{aligned}
 \neg \forall x \text{ in } Int[p(x) \Rightarrow q(x)] &\equiv \exists x \text{ in } Int[\neg(p(x) \Rightarrow q(x))] \text{ (extended De Morgan(i))} \\
 &\equiv \exists x \text{ in } Int[\neg(\neg p(x) \vee q(x))] \text{ (replacing } \Rightarrow) \\
 &\equiv \exists x \text{ in } Int[\neg \neg p(x) \wedge \neg q(x)] \text{ (De Morgan)} \\
 &\equiv \exists x \text{ in } Int[p(x) \wedge \neg q(x)] \text{ (negation(iii))}
 \end{aligned}$$

("It is not always true that $p(x)$ implies $q(x)$ " is equivalent to "There is a value of x with $p(x)$ true and $q(x)$ false".)

Solution 4.1

Suitable simple propositions are:

cat: This animal is a cat.

barks: This animal barks.

We can write the premises symbolically as below.

$$\begin{aligned}
 cat \Rightarrow \neg barks &\quad (1) \\
 barks &\quad (2)
 \end{aligned}$$

The conclusion is $\neg cat$.

A proof that the argument is valid can be given as follows.

$$\begin{array}{ll}
 cat \Rightarrow \neg barks & (1) \text{ premise} \\
 barks & (2) \text{ premise} \\
 \neg \neg barks & (3) \text{ Equivalent to (2) using negation(iii)} \\
 \neg cat & (4) \text{ contrapositive rule 1,3 (with } \neg barks \text{ for } t) \\
 \text{Hence} & \frac{cat \Rightarrow \neg barks; barks}{\neg cat}.
 \end{array}$$

Solution 4.2

The appropriate truth table is given below. Since the last column is always **true** we have a tautology. Hence the rule of inference given as disjunction is valid.

s	t	$(\neg s \wedge (s \vee t)) \Rightarrow t$
T	T	T
T	F	T
F	T	T
F	F	T

giving

s	t	$(\neg s \wedge (s \vee t)) \Rightarrow t$
T	T	T
T	F	T
F	T	T
F	F	T

Solution 4.3

We can use the following simple propositions.

c : This animal is a cat.

l : This animal likes dogs.

We can write the premises symbolically as below.

$$c \Rightarrow \neg l$$

$$\neg c$$

The conclusion is l .

To show that the argument is not valid, we draw up the truth table for $((c \Rightarrow \neg l) \wedge \neg c) \Rightarrow l$.

c	l	$((c \Rightarrow \neg l) \wedge \neg c) \Rightarrow l$
T	T	T
T	F	T
F	T	T
F	F	F

giving

c	l	$((c \Rightarrow \neg l) \wedge \neg c) \Rightarrow l$
T	T	T
T	F	T
F	T	T
F	F	F

The shaded row shows that the Boolean expression can be **false**, and so is not a tautology. Hence the argument is not valid.

Solution 4.4

We can use the following simple propositions.

$music$: I like this music.

$beaut$: This music is beautiful.

$record$: This music has been recorded.

The given propositions are then

$$music \quad (1)$$

$$\neg beaut \Rightarrow \neg music \quad (2)$$

$$\neg record \Rightarrow \neg beaut \quad (3)$$

$$\neg record \quad (4)$$

We can make deductions based on these premises as below.

$music$	(1) premise
$\neg beaut \Rightarrow \neg music$	(2) premise
$\neg record \Rightarrow \neg beaut$	(3) premise
$\neg record$	(4) premise
$\neg beaut$	(5) implication 3,4
$\neg music$	(6) implication 2,5
$music \wedge \neg music$	(7) conjunction 1,6
false	(8) equivalence to 7 by negation(ii)
Hence	$\frac{music; \neg beaut \Rightarrow \neg music; \neg record \Rightarrow \neg beaut; \neg record}{\text{false}}$

This shows that the given propositions are contradictory. (When working towards a contradiction, it is common practice to aim first to prove for some proposition a that both a and $\neg a$ must be **true**.)

Index

$\frac{s_1; s_2; \dots; s_n}{t}$, 38

$\exists$, 18

$\forall$, 15

argument, 35
 conclusion, 35
 premises, 35
 valid, 35

binding term, 15

Boolean algebra, 26

 rules of, 27
 absorption, 29
 associativity, 27
 commutativity, 27
 De Morgan's laws, 30
 distributivity, 27
 idempotency, 28
 negation, 29
 rewriting implication, 31

bound variable, 15

conclusion, 35

consistent, 45

contradiction, 45

De Morgan's laws, 30
 for quantifiers, 32

domain of interest, 5
 complete information, 7

equivalences with quantifiers
 commutativity, 32
 extended De Morgan, 32
 existential quantifier, 18

free variable, 15

inference, rules of , 38

 addition, 42
 combining $\vee, \Rightarrow$, 41
 conjunction, 42
 contrapositive, 39
 disjunction, 42
 equivalence, 42
 implication, 38
 simplification, 40
 transitivity, 42

modus ponens, 43

modus tollens, 43

predicate, 4

 argument of, 6
 arity, 6

premises, 35

proof, 38

quantifier, 14

 existential , 18
 universal, 15

scope, 15

semantic consequence, 35

syllogism, 35

syntax
 predicate calculus, 20

tautology, 30

universal quantifier, 15

universe of discourse, 5

valid argument, 35